

MORPHOIA / SPÉCIFICATION ET PROTOTYPE DE RÉFÉRENCE

MORPHOIA

Phase 2 - Candidat de standard expérimental 0.1

Reconstruction, modification, validation et génération de modèles CAO
paramétriques par l'Homme et par l'IA

Projet et auteur : MORPHOIA - Olivier Ami

Version du document : 0.1 expérimentale

Date : 3 août 2026

Autorité d'échange visée : STEP AP242 et ressources ISO 10303

Statut : proposition testable, non gelée et non normative

La stabilisation est interdite avant exécution P1/P2 sur deux backends, publication de la
conformité et bénéfice industriel mesuré.

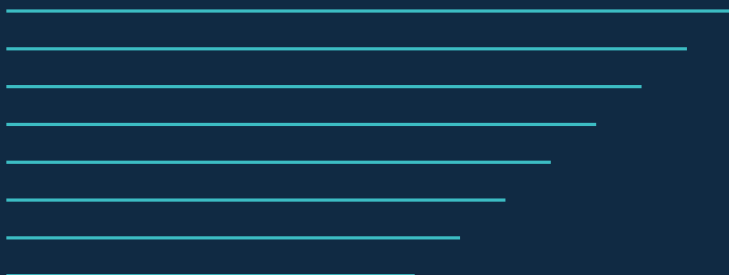


Table des matières

Table des matières	2
MORPHOIA - Phase 2	15
Statut	15
Livrables	15
Implémentation de référence	16
Auteur	16
Cahier des charges traçable	17
1. Règle de conception	17
2. Gates de standardisation	17
3. Exigences de gouvernance et de périmètre	17
4. Exigences sémantiques	18
5. Exigences d'exécution et de géométrie	18
6. Identité topologique et tolérances	18
7. PMI, assemblages et provenance	19
8. IA, sécurité et GPU	19
9. Interopérabilité et validation	19
10. Component Justification Record	19
Architecture de référence	21
1. Décision synthétique	21
2. Alternatives comparées	21
3. Couches et responsabilités	22
3.1 Sources et preuves	22
3.2 Extraction multimodale	22
3.3 Graphe canonique	22
3.4 Runtime	22
3.5 Noyaux et solveurs	22
3.6 Validation	22
3.7 Sorties	23

4. Double représentation	23
5. Identité et topological naming	23
6. Transactions, undo/redo et versioning	23
7. Déterminisme	23
8. Système d'extensions	24
9. MLIR, LLVM, OpenXLA et ONNX	24
10. Frontières de confiance	24
11. Composants réellement nouveaux	24
Spécification technique de la façade MORPHOIA 0.1	25
1. Portée et statut	25
2. Objectifs	25
3. Unité de compilation	25
4. Modèle d'exécution	25
4.1 Pureté	25
4.2 Affectation unique	25
4.3 Ordre	26
4.4 Atomicité	26
5. Déclarations	26
5.1 Paramètre et constante	26
5.2 Tolérances	26
5.3 Datums et systèmes de coordonnées	26
5.4 Esquisse	26
5.5 Feature	26
5.6 Référence topologique	27
5.7 Matériau, PMI et assemblage	27
5.8 Validation et scénarios	27
6. Système de types	27
6.1 Types fondamentaux	27
6.2 Unités	28
6.3 Types topologiques	28
7. Identifiants	28

8. Expressions	28
9. Attributs, provenance et incertitude	28
10. Profils	29
11. Extensions	29
12. Compilation	29
13. Graphe canonique JSON 0.1	29
14. Erreurs minimales	30
15. Sécurité et limites	30
16. Critères de maintien de la façade	30
Grammaire EBNF complète	31
Catalogue d'opérations candidate	33
1. Règles communes	33
2. Esquisses, contraintes et dimensions	33
Sketch	33
Constraint	33
Dimension	33
3. Features volumiques P1	33
Extrude	33
Cut	34
Revolve	34
Boolean	34
Hole	34
Pattern	34
Mirror	34
Fillet	34
Chamfer	34
4. Géométrie avancée P4	34
Sweep	34
Loft	34
Blend	35
Shell	35

Draft	35
5. Références et construction	35
Datum	35
Coordinate System	35
Reference Plane	35
Reference Axis	35
6. P2 - matériau, tolérances et PMI	35
Material	35
Thread	35
Tolerance	35
PMI	35
7. P3 - Assembly	35
8. Mécanisme d'ajout d'opérations	36
9. Matrice de réemploi	36
SDK et API MORPHOIA	37
1. Statut et portée	37
Légende de maturité	37
2. Principes imposés par la Phase 1	37
3. Inventaire exact du prototype Python	38
4. API Python réellement disponible	38
4.1 Validation sûre	39
4.2 Compilation candidate	39
4.3 AST et diagnostics	39
4.4 Références topologiques	39
4.5 Backend Python minimal	39
4.6 Transactions, provenance et pertes	40
5. CLI disponible	40
6. Architecture cible du SDK	40
6.1 Coeur C++ et ABI de plugins	40
6.2 Manifest backend	40
6.3 API cible par responsabilité	41
6.4 Backends cibles	41

6.5 Services cibles	41
7. Erreurs, pertes et diagnostics	41
8. Versioning et compatibilité	42
9. Sécurité cible	42
10. Gates d'implémentation SDK	42
Validation et conformité MORPHOIA	43
1. Statut	43
2. Contrôles réellement implémentés	43
3. Niveaux de validation cibles	44
4. Modèle de verdict	44
5. Validation du graphe	44
6. Validation géométrique cible	44
7. Validation comportementale cible	45
8. Registre des pertes	45
9. Qualification d'un backend	45
10. Profils candidats	46
11. Corpus et séparation des données	46
12. Gates mesurables	46
13. Performance et reproductibilité	47
14. Commandes disponibles aujourd'hui	47
15. Prochain incrément vérifiable	47
Interopérabilité, import et export	48
1. Principe	48
2. STEP	48
STEP AP203	48
STEP AP214	48
STEP AP242	48
ISO 10303-55/-108/-109/-111/-112/-113	48
3. Formats géométriques historiques	48
IGES	48

BREP	49
Parasolid XT	49
ACIS SAT/SAB	49
4. Noyaux et programmabilité ouverte	49
Open CASCADE Technology	49
FreeCAD	49
CadQuery	49
OpenSCAD	49
FeatureScript	49
5. CAO commerciales	50
Fusion 360	50
SolidWorks	50
CATIA	50
Creo	50
Siemens NX	50
Autodesk Inventor	50
Rhino et Grasshopper	50
6. DCC et formats de scène	50
Blender	50
glTF	51
USD et OpenUSD	51
Siemens JT	51
7. BIM et IFC	51
8. Meshes et captures	51
9. Fabrication et qualité	51
10. SDK de traduction	51
11. Compilateurs directs	51
12. Compilateurs inverses	52
13. Matrice de qualification	52
Intelligence artificielle et accélération GPU	53
1. Principe de sûreté	53
2. Architectures comparées	53

3. Pipeline multimodal	53
3.1 Dessins industriels et vues orthographiques	53
3.2 Croquis manuscrits	53
3.3 Photographies et vidéo	54
3.4 Scans et nuages de points	54
3.5 STL, OBJ, PLY et meshes	54
3.6 STEP, IGES, Parasolid et ACIS	54
3.7 Texte et voix	54
4. Génération contrainte	54
5. Objectifs d'apprentissage	54
6. Données nécessaires	55
6.1 Unité de donnée	55
6.2 Corpus	55
6.3 Sources existantes	55
7. Entraînement	55
Etape A - préentraînement	55
Etape B - supervision structurée	55
Etape C - exécution dans la boucle	55
Etape D - préférences ingénieur	55
Etape E - calibration	56
8. Métriques IA	56
9. Carte CPU/GPU	56
10. Technologies GPU	56
CUDA et Tensor Cores	56
RTX et OptiX	56
Vulkan Compute	56
DirectML	56
OpenCL	57
ROCm	57
SYCL/oneAPI	57
11. Estimations de performance	57
12. Portabilité ML	57

13. Sécurité MLOps	57
Guide utilisateur de MORPHOIA	58
1. Statut à lire avant toute utilisation	58
2. Ce que le prototype sait réellement faire	58
Implémenté	58
Pas encore implémenté	58
3. Installation	59
3.1 Prérequis	59
4. Premier parcours	59
4.1 Valider l'exemple fourni	59
4.2 Compiler vers le graphe canonique JSON	59
4.3 Obtenir des diagnostics lisibles par une machine	60
5. Écrire un document 0.1	60
5.1 En-tête, namespace et modèle	60
5.2 Paramètres et unités	60
5.3 Les quatre tolérances non interchangeable	60
5.4 Esquisse et feature	60
5.5 Référence topologique persistante	61
5.6 Provenance et incertitude	61
5.7 Scénarios d'édition	61
6. Profils actuels	61
7. Comprendre les diagnostics	62
8. Utiliser l'API Python	62
9. Bonnes pratiques pendant la phase expérimentale	62
10. Gates hérités de la Phase 1	63
11. Documents associés	63
Guide développeur de MORPHOIA	64
1. Portée et statut	64
2. Préparer l'environnement	64
3. Arborescence du prototype	64
4. Pipeline réellement exécuté	65

5. Modules et responsabilités	65
5.1 model.py	65
5.2 lexer.py	65
5.3 parser.py	65
5.4 typesystem.py	66
5.5 semantic.py	66
5.6 compiler.py	66
5.7 topology.py	67
5.8 provenance.py	67
5.9 losses.py	67
5.10 runtime.py	67
5.11 backends/	68
6. API Python publique actuelle	68
7. CLI actuelle	68
8. Ajouter ou modifier une construction syntaxique	68
9. Ajouter une unité ou une fonction typée	69
10. Ajouter une opération à un profil	69
11. Ajouter un backend	69
12. Tests	70
13. Diagnostics	70
14. Frontières à ne pas franchir silencieusement	71
15. Component Justification Record	71
16. Gates de développement	71
Tutoriels et FAQ de MORPHOIA	72
Préambule	72
Tutoriel 1 - Valider et compiler l'exemple officiel	72
Étape 1 : installer le checkout	72
Étape 2 : vérifier la version du paquet	72
Étape 3 : valider	72
Étape 4 : compiler vers l'IR	72
Étape 5 : inspecter le JSON	72

Tutoriel 2 - Écrire une pièce minimale	72
Ajouter une contrainte de domaine	73
Tutoriel 3 - Comprendre une erreur d'unité	73
Tutoriel 4 - Déclarer une référence persistante	73
Observer l'échec volontaire	73
Tutoriel 5 - Détecter un cycle de dépendances	74
Tutoriel 6 - Utiliser l'API Python	74
Tutoriel 7 - Tester les trois états d'une référence	74
Tutoriel 8 - Vérifier le déterminisme du prototype	75
Tutoriel 9 - Utiliser une transaction en mémoire	75
Tutoriel 10 - Enregistrer preuve, décision et perte	76
FAQ	76
MORPHOIA est-il déjà un nouveau standard CAO ?	76
Pourquoi le fichier dit-il 0.1 alors que la CLI affiche 0.2.0-dev ?	76
La commande compile produit-elle un modèle CAO ?	76
Puis-je ouvrir le JSON dans FreeCAD, CATIA, NX ou SolidWorks ?	76
Le dépôt contient-il Open CASCADE ?	76
Pourquoi ne pas écrire directement du CadQuery ?	77
Pourquoi STEP/AP242 n'est-il pas simplement remplacé ?	77
Les opérations extrude et fillet fonctionnent-elles ?	77
Pourquoi mon feature incomplet peut-il être déclaré valide ?	77
Quels profils existent ?	77
Pourquoi cardinality = unique est-il obligatoire ?	77
Puis-je utiliser cardinality = many ?	77
Les UUID sont-ils obligatoires ?	77
Le hash sémantique est-il définitif ?	77
Les imports sont-ils téléchargés ?	77
Les scénarios validate sont-ils exécutés ?	77
MORPHOIA possède-t-il déjà undo/redo et des transactions ?	78
Le registre des pertes est-il déjà utilisé par les exports ?	78
MORPHOIA utilise-t-il déjà l'IA ou le GPU ?	78
Peut-on générer des fichiers .morph avec un LLM ?	78

Quelle est la licence ?	78
Comment proposer un composant nouveau ?	78
Quels sont les gates avant une version 1.0 ?	78
Où continuer ?	78
Feuille de route, gouvernance et adoption	79
Statut de ce document	79
Principes de conduite	79
Profils de conformité proposés	79
Étapes et gates	79
Fondation - mois 0 à 3	79
0.1 POC - mois 3 à 12	80
0.3 Alpha - mois 12 à 18	80
0.5 MVP vertical - mois 18 à 30	80
0.8 Bêta - mois 30 à 42	80
1.0 - mois 42 à 60	81
Organisation cible	81
Organes	81
Processus de décision	81
Cycle de publication	81
Licences et politique de propriété intellectuelle	81
Stratégie d'adoption	82
Coûts, risques et analyse de faisabilité industrielle	83
Portée et méthode	83
Estimation par phase	83
Équipe MVP indicative	83
Coûts directs à ne pas sous-estimer	83
Registre des risques	84
Verrous scientifiques	84
Intention sous-déterminée	84
Topological Naming Problem	84
Équivalence procédurale	84

Robustesse géométrique	84
Validation IA	85
B-rep exacte sur GPU	85
Verrous industriels	85
Critères d'arrêt	85
Bénéfices industriels à mesurer	85
Références normatives et techniques de la Phase 2	86
Politique de citation	86
ISO 10303 et STEP	86
Autres standards et formats	87
Noyaux, CAO et SDK	87
Compilateurs et exécution	87
IA, vision et GPU	88
Interopérabilité et archivage	88
Références propres à MORPHOIA	88
ADR-0001 - Coeur STEP-centrique	90
Contexte	90
Décision	90
Conséquences	90
ADR-0002 - Façade textuelle expérimentale	91
Contexte	91
Décision	91
Critères de maintien	91
Condition d'abandon	91
ADR-0003 - OCCT comme backend ouvert de référence	92
Contexte	92
Décision	92
Conséquences	92
CJR-XXXX - Nom du composant	93

Métadonnées	93
Besoin non couvert	93
Solutions existantes évaluées	93
Protocole reproductible	93
Résultat	93
Interopérabilité	93
Coût et risques	93
Stratégie de remplacement	93
Décision et condition d'abandon	93

MORPHOIA - Phase 2

Statut

Cette branche documentaire constitue le Draft expérimental 0.1 de MORPHOIA. Elle ne constitue ni une norme ISO, ni une spécification industrielle stable, ni une promesse de compatibilité universelle.

La Phase 1 interdit de figer immédiatement un nouveau langage public. En conséquence, MORPHOIA 0.1 est organisé comme :

1. un profil exécutable, borné et STEP-centrique ;
2. un graphe canonique interne versionné ;
3. une façade textuelle expérimentale destinée à l'humain et à l'IA ;
4. une implémentation de référence permettant de mesurer cette façade ;
5. une suite de validation appelée à devenir une suite de conformité.

Le statut normatif ne pourra être proposé qu'après trois preuves cumulatives :

- profils P1 et P2 exécutés par au moins deux backends indépendants ;
- suite de conformité publique ;
- bénéfice industriel mesuré sur des pilotes représentatifs.

Livrables

Document	Objet
[Exigences](requirements.md)	Conclusions de la Phase 1 transformées en exigences opposables
[Architecture](architecture.md)	Alternatives, choix et responsabilités de chaque couche
[Spécification du langage](language-specification.md)	Modèle d'exécution, types, sémantique et extensions
[Grammaire EBNF](grammar.ebnf)	Grammaire candidate complète de la façade textuelle 0.1
[Catalogue des opérations](operation-catalog.md)	Contrats minimaux des opérations P1 à P4
[Interopérabilité](interoperability.md)	Import, export, pertes et rôle de chaque standard ou CAO
[IA et GPU](ai-gpu.md)	Pipeline multimodal, entraînement, accélération et limites
[SDK et API](sdk-api.md)	API Python, coeur C++, plugins et services
[Validation](validation-conformance.md)	Validateurs, benchmarks, propriétés et critères de passage
[Manuel utilisateur](user-guide.md)	Installation, écriture, validation et compilation
[Manuel développeur](developer-guide.md)	Structure du code, conventions, tests et ajout de composants
[Roadmap et gouvernance](roadmap-governance.md)	MVP, Alpha, Bêta, 1.0, licences et adoption
[Coûts et risques](costs-risks.md)	Estimations, scénarios et plans de réduction du risque
[Références](references.md)	Normes et documents primaires

Implémentation de référence

La série 0.1 fournit un lexer, un parser, un validateur sémantique, un compilateur vers le graphe canonique JSON et un résolveur de références topologiques à trois états. Elle ne fournit pas encore un noyau géométrique ni un export STEP : ces fonctions doivent réutiliser OCCT et les ressources STEP, conformément à la Phase 1.

```
PYTHONPATH=src python -m morphoia validate examples/mounting_plate.morph
PYTHONPATH=src python -m morphoia compile examples/mounting_plate.morph \
-o build/mounting_plate.mcir.json
PYTHONPATH=src python -m unittest discover -s tests -v
```

Auteur

MORPHOIA a été initié par Olivier Ami.

Cahier des charges traçable

1. Règle de conception

Toute décision respecte la règle suivante :

Un composant nouveau n'est autorisé que si une analyse reproductible démontre qu'aucune solution existante ne satisfait les exigences, ou si un benchmark préenregistré démontre un avantage mesurable de performance, robustesse, interopérabilité ou simplicité.

Cette règle s'applique au langage, au runtime, aux schémas, aux solveurs, aux noyaux, aux bibliothèques GPU, aux modèles d'IA et aux formats d'échange.

2. Gates de standardisation

Gate	Preuve requise	Statut initial
G0 - Draft expérimental	Spécification, implémentation de référence, 100 cas golden	En cours
G1 - Profil portable	P1 et P2 sur deux backends, validité B-rep au moins 99 %, scénarios d'édition au moins 95 %	Non atteint
G2 - Conformité publique	Corpus, propriétés de validation, registre des pertes et implémentation indépendante	Non atteint
G3 - Valeur industrielle	Gain médian de temps humain au moins 40 % sur deux pilotes	Non atteint
G4 - Standard candidate	G1, G2 et G3, gouvernance externe et politique brevets	Interdit avant preuves

Les artefacts 0.x doivent porter les mots `experimental`, `draft` ou `candidate`. Les mots `standard`, `stable`, `certified` et `1.0` sont réservés aux gates correspondants.

3. Exigences de gouvernance et de périmètre

ID	Exigence	Critère d'acceptation
GOV-001	Réutiliser avant de créer.	Chaque composant possède une Component Justification Record, ou une référence explicite à une brique existante.
GOV-002	Ne pas figer le DSL avant G4.	La syntaxe 0.x reste expérimentale et remplaçable.
GOV-003	Comparer sur le même corpus.	Alternatives, versions, licences et conditions de benchmark sont enregistrées.
GOV-004	Séparer spécification, implémentation et certification.	Aucun succès du runtime de référence ne vaut preuve de conformité universelle.
SCP-001	Commencer par P1.	Esquisses, extrusion, révolution, trou, booléen, motif, miroir, congé et chanfrein constants.
SCP-002	Versionner P1, P2, P3 et P4 séparément.	Chaque opération déclare son profil minimal et son fallback.
SCP-003	Refuser explicitement le hors-profil.	Au moins 95 % des pièces hors profil sont refusées ou dirigées vers revue.

4. Exigences sémantiques

ID	Exigence	Critère d'acceptation
SEM-001	Ancrer le modèle dans AP242 et ISO 10303-42/-55/-108/-109/-111/-112/-113.	Chaque concept possède un mapping ou un écart documenté.
SEM-002	Ne pas recréer une ontologie concurrente.	Les extensions sont namespacées et limitées aux écarts démontrés.
SEM-003	Conserver procédure et résultat explicite.	Toute régénération validée lie le graphe, la B-rep, les propriétés et l'environnement.
SEM-004	Employer un graphe typé.	Tous les noeuds et relations nécessaires à la régénération sont explicites.
SEM-005	Déclarer support, approximation, fallback, perte ou refus.	Aucune dégradation silencieuse.
SEM-006	Produire un hash sémantique canonique.	Espaces, commentaires et ordre non significatif ne modifient pas le hash.
SEM-007	Assurer la pérennité.	Le graphe et la dernière représentation explicite restent lisibles sans SDK propriétaire.

5. Exigences d'exécution et de géométrie

ID	Exigence	Critère d'acceptation
GEO-001	Utiliser OCCT comme backend exact ouvert de référence.	Aucun type OCCT ne fuit dans le contrat canonique.
GEO-002	Permettre des backends substituables.	Interface de capacités commune et tests différentiels.
GEO-003	Ne pas réécrire un noyau B-rep.	Aucun budget POC/MVP alloué à un noyau généraliste nouveau.
EXE-001	Verrouiller l'environnement.	Versions du runtime, noyau, solveur, extensions et options sont enregistrées.
EXE-002	Déterminisme sémantique.	Même source validée produit le même graphe et le même hash.
EXE-003	Déterminisme du profil.	Même graphe et même lockfile produisent les mêmes propriétés de validation.
EXE-004	Transaction et rollback.	Un échec ne remplace jamais le dernier état valide.
EXE-005	DAG explicite.	Cycles de construction refusés avant exécution.
SOL-001	Réutiliser un solveur existant.	FreeCAD Sketcher, SolveSpace ou D-Cubed évalué avant toute implémentation nouvelle.
SOL-002	Capturer la sémantique du solveur.	Branche, initialisation, priorités, degrés de liberté, résidus et diagnostics enregistrés.

6. Identité topologique et tolérances

ID	Exigence	Critère d'acceptation
REF-001	Interdire les indices natifs comme identité persistante.	Toute référence combine producteur, rôle, lignée et preuves géométriques ou d'adjacence.
REF-002	Résolution à trois états.	unique, ambiguous ou missing; zéro ambiguïté silencieuse.
REF-003	Tester après édition.	Variation, suppression et réordonnement pertinents sont testés.
REF-004	Conserver l'UUID de l'intention.	Renommer un symbole ne change pas son UUID explicite.
TOL-001	Séparer quatre tolérances.	Tolérance noyau, incertitude capteur, tolérance dimensionnelle et zone GD&T sont des types distincts.
TOL-002	Interdire le healing silencieux.	Toute réparation est un événement et une perte potentielle.

7. PMI, assemblages et provenance

ID	Exigence	Critère d'acceptation
PMI-001	Porter le PMI sémantique.	Datums, zones, modificateurs, unités et associations survivent aux scénarios P2.
ASM-001	Séparer définition et occurrence.	Identité produit, occurrence, placement et configuration restent distincts.
PROV-001	Relier chaque inférence à une preuve.	Source, région, cote, règle, modèle et confiance sont référencés.
PROV-002	Représenter l'incertitude.	certain, hypothesis, ambiguous et unknown sont distincts.
LOSS-001	Produire un registre des pertes.	Chaque import, exécution et export fournit un rapport machine-readable.

8. IA, sécurité et GPU

ID	Exigence	Critère d'acceptation
AI-001	L'IA propose, le runtime valide.	Aucune sortie IA n'est autoritative avant validations déterministes.
AI-002	Autoriser plusieurs hypothèses et l'abstention.	Une dimension absente n'est jamais inventée comme vérité.
AI-003	Décodage contraint.	Au moins 98 % de sorties syntaxiquement valides constitue une cible expérimentale.
DAT-001	Employer des données à droits démontrés.	Licence, consentement contractuel et usages permis attachés à chaque actif.
SEC-001	Interdire les effets de bord dans la source.	Pas d'I/O, réseau, horloge, aléa, récursion ou boucle générale.
SEC-002	Isoler les adaptateurs non fiables.	Quotas CPU, GPU, mémoire, temps et système de fichiers.
GPU-001	GPU pour perception et calcul dense.	Vision, OCR, points, rendu, distances, tessellation et IA priorisés.
GPU-002	CPU/noyau exact pour l'autorité initiale.	Booléens, fillets, solveur, healing et PMI restent qualifiés sur CPU.
GPU-003	Prouver tout portage.	Temps p50/p95, mémoire, précision, déterminisme et taux d'échec comparés au CPU.

9. Interopérabilité et validation

ID	Exigence	Critère d'acceptation
INT-001	AP242 comme autorité d'échange mécanique.	Round-trip géométrie et propriétés au moins 99 % sur le profil supporté.
INT-002	AP238/QIF pour fabrication et qualité.	Liens explicites vers les entités canoniques.
INT-003	JT, glTF et OpenUSD comme vues dérivées.	Une vue maillée ne remplace jamais l'autorité paramétrique.
INT-004	IGES, STL, OBJ et PLY comme frontières héritées.	Informations absentes et reconstruction requise sont déclarées.
ADP-001	Isoler les CAO propriétaires.	Matrice CAO x version x profil, sans dépendance fournisseur dans le coeur.
VAL-001	Séparer les niveaux de validation.	Syntaxe, sémantique, contraintes, géométrie, topologie, comportement et industrie ont des verdicts distincts.
VAL-002	Mesurer les scénarios d'édition.	Au moins 95 % réussis sur les pièces acceptées.
VAL-003	Dimensions explicites exactes ou abstention.	100 % sur les cas automatiquement acceptés.
VAL-004	Conservation des unités et tolérances.	100 %.
VAL-005	Validité B-rep.	Au moins 99 % sans healing silencieux.

10. Component Justification Record

Tout composant nouveau doit posséder un fichier CJR-XXXX.md contenant :

1. exigences non couvertes ;
2. solutions existantes et versions examinées ;
3. couverture, architecture, performance, licence et pérennité ;
4. corpus et protocole reproductibles ;
5. échec démontré ou avantage quantifié ;

6. coût de développement et de maintenance ;
7. mapping STEP et pertes ;
8. stratégie de remplacement ;
9. décision, approbateurs et date ;
10. condition d'abandon si l'avantage n'est pas confirmé.

Il n'existe pas de seuil universel valable pour tout composant. La métrique doit être préenregistrée : temps, mémoire, taux de succès, robustesse, portabilité, pertes sémantiques, tokens ou temps humain.

Architecture de référence

1. Décision synthétique

MORPHOIA adopte une architecture hybride, STEP-centrique et multi-backend. Le coeur n'est ni un nouveau noyau, ni un format maillé, ni une IA générative. Il s'agit d'une couche de confiance qui relie :

- preuves d'entrée et provenance ;
- graphe de construction typé ;
- exécution déterministe ;
- noyaux et solveurs existants ;
- résultat B-rep explicite ;
- validation comportementale ;
- registre des pertes ;
- sorties spécialisées.

```

Sources immuables
plans | images | texte | scans | meshes | STEP | CAO native
    |
    v
Extraction multimodale et graphe de preuves
    |
    v
Candidats IA classes + confiance + abstention
    |
    v
Graphe canonique STEP-centrique
    |
    v
Runtime déterministe et transactionnel
    |
    +--> solveur existant
    +--> OCCT reference ouverte
    +--> FreeCAD oracle ouvert
    +--> Parasolid/ACIS/CAO en option
    |
    v
B-rep + PMI + propriétés + registre des pertes
    |
    +--> AP242 / AP238 / QIF
    +--> FreeCAD / CAO natives
    +--> JT / glTF / OpenUSD / Blender
  
```

2. Alternatives comparées

Stratégie	Avantage	Limite décisive	Décision
IA vers fichier natif	Démonstration rapide	Verrou fournisseur, faible vérifiabilité	Adaptateur seulement
STEP seul sans runtime	Pérennité et richesse normative	Sémantique d'exécution peu implémentée	Autorité sémantique, insuffisante seule
CadQuery ou Python comme standard	Productivité et OCCT	Langage général, effets de bord, PMI incomplet	SDK et frontend de comparaison
FeatureScript	Features et requêtes riches	Runtime propriétaire Onshape	Référence de conception
MLIR comme format public	Typage, passes et dialectes	Pas de sémantique mécanique intrinsèque	IR de compilation optionnelle
Mesh/implicite comme autorité	GPU, scan et formes complexes	Perte d'intention, PMI et B-rep	Représentation complémentaire
Nouveau noyau exact	Contrôle complet	50-150 M€, 7-12 ans, risque extrême	Rejeté
Façade textuelle + graphe STEP	Lisibilité, sûreté, IA et Git	Adoption à démontrer	Draft 0.x falsifiable

3. Couches et responsabilités

3.1 Sources et preuves

Chaque source est immuable et identifiée par URI, type MIME, empreinte, unité connue ou inconnue, licence et règles d'usage. Une preuve peut viser une cote, une région d'image, un primitif de nuage de points, une entité STEP, une phrase ou une décision humaine.

Cette couche réutilise PDF, DXF, STEP, JT, QIF et les SDK de traduction. MORPHOIA n'invente pas un conteneur multimédia.

3.2 Extraction multimodale

La vision, l'OCR et les encodeurs 3D produisent des observations, jamais des vérités géométriques. Ils peuvent proposer plusieurs candidats et doivent calibrer leur confiance. Les sorties restent séparées du graphe accepté.

3.3 Graphe canonique

Le graphe porte produit, paramètres, opérations, contraintes, références, PMI, configurations, preuves et scénarios. Son vocabulaire est mappé aux ressources ISO 10303. Le JSON 0.1 n'est qu'un encodage testable ; l'abstract data model constitue le contrat.

La structure de lecture est un arbre. Les dépendances de calcul forment un DAG. Les contraintes forment un hypergraphe séparé qui peut contenir des cycles algébriques.

3.4 Runtime

Le runtime effectue :

1. parsing et résolution des versions ;
2. typage et unités ;
3. construction du DAG ;
4. vérification des capacités ;
5. résolution des contraintes ;
6. exécution des opérations ;
7. propagation des lignées ;
8. résolution des références ;
9. validation géométrique et PMI ;
10. scénarios d'édition ;
11. commit atomique ou rollback.

Le runtime de confiance n'exécute jamais de Python arbitraire généré par IA.

3.5 Noyaux et solveurs

OCCT fournit le backend exact ouvert initial. OCAF/XDE sont réutilisés pour documents, transactions, structure produit et échange STEP. FreeCAD apporte un oracle de recompute et un solveur d'esquisse ouvert. Parasolid, ACIS, CGM, C3D et les CAO natives restent des plugins optionnels soumis à licence.

La frontière de plugin expose des types MORPHOIA et des handles opaques. Aucun pointeur ou identifiant natif de noyau n'est sérialisé comme identité canonique.

3.6 Validation

La validation est indépendante de l'exécution : un backend peut construire un solide mais échouer la conformité comportementale. Chaque rapport contient verdicts, preuves, versions, propriétés, ambiguïtés, réparations et pertes.

3.7 Sorties

AP242 est la sortie mécanique neutre privilégiée. AP238 et QIF couvrent fabrication et qualité. JT, glTF et OpenUSD restent des vues dérivées. Les formats natifs sont produits par des adaptateurs versionnés et ne deviennent jamais une dépendance du coeur.

4. Double représentation

Chaque révision validée conserve :

- le graphe procédural ;
- la représentation explicite exacte ;
- les propriétés de validation ;
- l'environnement verrouillé ;
- le lien entre entités canoniques et entités du résultat ;
- le registre des pertes.

Ce principe reprend ISO 10303-55. La B-rep permet d'ouvrir et d'inspecter le dernier état même si une opération future n'est plus exécutable. Le graphe reste l'autorité d'édition.

5. Identité et topological naming

Une identité persistante n'est pas un index de face. Elle comprend :

1. UUID de l'intention ;
2. feature productrice ;
3. rôle de sortie ;
4. lignée de création, modification, fusion ou scission ;
5. filtre géométrique typé ;
6. contexte d'adjacence ;
7. signature de vérification ;
8. cardinalité attendue.

La résolution retourne unique, ambiguous ou missing. Un score peut ordonner les candidats pour l'affichage, mais ne transforme jamais une ambiguïté en choix automatique.

6. Transactions, undo/redo et versioning

Chaque modification crée une transaction comprenant source, paramètres modifiés, DAG affecté, résultats, propriétés, logs et verdicts. Le commit n'a lieu qu'après les validations exigées par le profil. Undo et redo rejouent des révisions immuables ; ils ne tentent pas d'inverser numériquement une opération.

Git versionne les sources, spécifications et manifests. Les gros résultats B-rep peuvent être placés dans un stockage de contenu adressé par hash ou Git LFS. Le hash sémantique ignore les détails de mise en forme mais couvre chaque décision géométrique.

7. Déterminisme

Trois niveaux sont distingués :

- déterminisme sémantique : même graphe et mêmes décisions ;
- déterminisme d'un profil : même noyau, solveur, versions et options ;
- équivalence multi-backend : propriétés dans les tolérances, sans exiger les mêmes octets.

Les opérations parallèles indépendantes peuvent être calculées simultanément. Leur ordre de commit est fixé par le DAG puis l'UUID. Les réductions GPU non déterministes ne participent pas à une propriété autoritative sans mode qualifié.

8. Système d'extensions

Une extension est identifiée par URI, version exacte, empreinte, licence, types, opérations, effets, rôles topologiques, mapping STEP, fallback et suite de tests. Une extension inconnue peut être préservée, mais pas exécutée.

Le code natif arbitraire n'est jamais embarqué dans un document. Les implémentations sont des plugins signés ou, après expérimentation, des modules WebAssembly sans I/O dans un profil déterministe.

9. MLIR, LLVM, OpenXLA et ONNX

MLIR est approprié pour représenter opérations, types, attributs, interfaces et passes. Un dialecte interne MORPHOIA pourra accélérer la vérification et le lowering C++/GPU. Il ne devient pas le format d'archive : il ne porte pas nativement produit, PMI, provenance ou identité topologique.

LLVM compile le runtime et les extensions sûres. ONNX Runtime, OpenXLA, IREE et TVM déploient les modèles IA. Aucun de ces outils ne définit la sémantique CAO.

10. Frontières de confiance

Zone	Confiance	Mesures
Source validée	Haute après validation	Schéma, types, unités, DAG, quotas
Modèle IA	Non fiable	Décodage contraint, abstention, provenance
Parser de format externe	Non fiable	Processus isolé, limites, fuzzing
Backend ouvert qualifié	Conditionnelle	Version pinning, corpus, propriétés
SDK propriétaire	Conditionnelle	Worker isolé, contrat de licence, matrice de versions
Vue glTF/USD/JT	Dérivée	Jamais autorité paramétrique

11. Composants réellement nouveaux

La Phase 1 justifie seulement :

- profil exécutable des ressources STEP procédurales ;
- runtime neutre de référence ;
- mapping de features vers les backends avec pertes ;
- référence persistante multi-stratégie avec abstention ;
- validateur comportemental et différentiel ;
- graphe de preuves et d'hypothèses ;
- registre de pertes ;
- suite de conformité paramétrique.

La façade `.morph` est un composant expérimental. Elle doit être abandonnée si elle n'améliore pas de manière mesurable la validité IA, la sécurité ou le temps humain par rapport aux alternatives.

Spécification technique de la façade MORPHOIA 0.1

1. Portée et statut

Ce document spécifie la façade textuelle expérimentale `.morph` et son lowering vers le MORPHOIA Canonical Construction Graph, ou MCCG. Le terme **MUST** indique une règle vérifiée par une implémentation conforme au draft. Le terme **SHOULD** indique une recommandation dont toute dérogation doit être documentée.

La façade n'est pas l'autorité normative future. Le modèle abstrait STEP-centrique, les profils d'exécution et la suite de conformité sont prioritaires. La syntaxe peut changer ou être abandonnée avant 1.0.

2. Objectifs

La source doit être :

- lisible par un ingénieur ;
- générable par une IA avec décodage contraint ;
- déterministe et sans état implicite ;
- non Turing-complète ;
- versionnable avec Git ;
- typée en unités et entités topologiques ;
- indépendante d'un noyau ou d'une CAO ;
- compilable vers un graphe canonique ;
- capable de préserver provenance, incertitude et pertes.

Elle ne décrit pas seulement une forme finale. Elle décrit une construction paramétrique et des critères permettant de la vérifier après modification.

3. Unité de compilation

Un document commence par la version de la façade :

```
morphoia 0.1;
namespace org.example.parts;
```

Il peut importer des extensions par URI ou chemin. Une implémentation de production **MUST** verrouiller version et empreinte dans un manifest externe :

```
import "urn:morphoia:extension:sheet-metal:0.3.1" as sheet;
```

Le document contient un ou plusieurs modèles. Le profil est un attribut du modèle :

```
model Bracket [profile = "P1", id = "uuid-stable"] {
  // declarations
}
```

4. Modèle d'exécution

4.1 Pureté

La série 0.1 ne possède ni I/O, ni réseau, ni horloge, ni générateur aléatoire, ni boucle générale, ni récursion. Les opérations sont pures : leurs résultats dépendent uniquement de leurs entrées, du profil d'exécution verrouillé et des décisions explicites.

4.2 Affectation unique

Une déclaration nommée ne peut pas être redéfinie dans le même modèle. Les scénarios peuvent appliquer des valeurs temporaires avec `set`, mais créent une transaction isolée. Ils ne modifient pas l'état de référence.

4.3 Ordre

L'ordre textuel sert à la lecture et au diff. L'ordre d'évaluation résulte du DAG des dépendances. Les cycles de construction sont des erreurs statiques. Les cycles algébriques internes au solveur de contraintes sont autorisés car ils appartiennent à un hypergraphe distinct.

4.4 Atomicité

Une régénération passe par validation, exécution, résolution des références, propriétés et scénarios. Un échec entraîne rollback. Le dernier snapshot valide reste disponible.

5. Déclarations

5.1 Paramètre et constante

```
parameter width: length = 80 mm [
  min = 40 mm,
  max = 200 mm,
  status = observed,
  evidence = "drawing:D1"
];
constant pi_value: scalar = 3.141592653589793;
```

Un paramètre peut être driving, measured, inferred ou reference via attribut. Une valeur inférée ou ambiguë MUST référencer au moins une preuve.

5.2 Tolérances

Les quatre types sont incompatibles :

```
tolerance build_precision: kernel_tolerance = 0.001 mm;
tolerance scan_precision: measurement_uncertainty = 0.05 mm;
tolerance drawing_size: dimensional_tolerance = 0.10 mm;
tolerance position_zone: gdt_zone = 0.20 mm;
```

Une tolérance dimensionnelle ne peut pas servir à réparer une intersection de noyau. Une incertitude capteur ne devient pas un intervalle fonctionnel sans décision explicite.

5.3 Datums et systèmes de coordonnées

```
datum base_plane: plane = world.xy;
datum axis_z: axis = world.z;
```

Le repère mécanique par défaut est cartésien et droitier. Une esquisse MUST nommer son plan. Aucun Workplane global implicite n'existe. Les transformations d'assemblage P3 sont rigides ; échelle et cisaillement sont interdits dans le profil mécanique.

5.4 Esquisse

```
sketch outline on base_plane {
  profile outer = rectangle(width, depth, centered = true);
  constraint horizontal(outer.bottom);
  constraint vertical(outer.left);
  dimension overall_width: length = width;
}
```

Le bloc porte entités, contraintes, dimensions et régions. L'implémentation de référence 0.1 accepte un noeud générique ; le profil d'exécution doit fournir un contrat de solveur : état de contrainte, branche, initialisation, seuils, redondances et diagnostics.

5.5 Feature

```
feature blank: extrude {
  profile = outline.outer;
  distance = thickness;
  direction = +world.z;
  result = solid;
}
```

Le type après : est l'opération qualifiée. Chaque argument influençant la géométrie MUST être présent ou défini par le profil avec une valeur normative. Les valeurs implicites propres à une CAO sont interdites.

Chaque feature produit une nouvelle version immuable du corps :

```
BodyVersion[n] + Feature[n+1] -> BodyVersion[n+1]
```

5.6 Référence topologique

```
reference top_face: face = select(
  blank.faces,
  role = "cap.end",
  normal = world.z,
  cardinality = unique
);
```

Une référence **MUST** utiliser `select`, porter une collection liée à la lignée, un rôle, une signature géométrique ou d'adjacence et une cardinalité unique lorsque le consommateur attend une entité. La résolution retourne :

- unique avec une entité ;
- ambiguous avec tous les candidats ;
- missing sans candidat.

Une implémentation **MUST NOT** choisir le premier candidat, même si un score permet de les classer.

5.7 Matériau, PMI et assemblage

```
material alloy: material = material_record("EN AW-6061 T6");
pmi hole_size: diameter = diameter_dimension(hole_wall, nominal = hole_diameter);

assembly product {
  occurrence left = instance(part_a);
  occurrence right = instance(part_b);
  constraint mate(left.axis, right.axis);
}
```

Les PMI graphiques ne suffisent pas. P2 exige cible, datum reference frame, zone, modificateurs, unités et règle générale. P3 distingue définition produit, occurrence, placement et configuration.

5.8 Validation et scénarios

```
validate {
  assert manifold(finished);
  assert resolves(top_face, unique);
  scenario WiderPlate {
    set width = 100 mm;
    expect valid(finished);
    expect resolves(top_face, unique);
  }
}
```

Un scénario décrit une modification et des invariants observables. Il est la base de la conformité comportementale.

6. Système de types

6.1 Types fondamentaux

Famille	Types
Logique	boolean
Numérique	integer, scalar, rational
Texte et identité	string, uri, uuid, digest, semver
Quantités	length, angle, area, volume, mass, time, temperature, force, pressure, density, ratio
Coordonnées	point2, vector2, direction2, point3, vector3, direction3, frame2, frame3, transform
Géométrie	curve2, curve3, surface, region2, sketch
Topologie	vertex, edge, wire, face, shell, solid, body_set
Produit	part, occurrence, assembly, configuration, material
MBD	datum, datum_system, pmi, geometric_tolerance, surface_texture
Provenance	evidence, hypothesis, decision, loss_record

La grammaire 0.1 exprime un `TypeRef` qualifié simple. Les types paramétrés comme `Ref<Face, one>` appartiennent au modèle abstrait cible et seront introduits uniquement après preuve d'ergonomie et de parsing indépendant.

6.2 Unités

Une quantité physique porte une unité : 25 mm, 0.5 deg. Les opérations arithmétiques vérifient les dimensions. `length + angle` est une erreur. `angle` n'est pas assimilé implicitement à `scalar`.

Le registre initial comprend SI et unités industrielles explicitement listées. Le hash sémantique utilise valeur canonique et dimension, tandis que la source conserve l'unité d'écriture. `NaN` et `Infinity` sont interdits.

6.3 Types topologiques

Une face, une arête ou un solide ne sont pas interchangeables. Une collection topologique non ordonnée ne peut pas être indexée pour créer une identité persistante. Les opérations doivent déclarer leurs rôles de sortie.

7. Identifiants

Une déclaration peut porter [`id` = "..."]. L'ID explicite reste stable après renommage. Si l'ID est absent dans le draft, le compilateur crée un UUIDv5 déterministe à partir du namespace et du chemin symbolique. Cette commodité est réservée au prototypage ; les artefacts destinés au round-trip SHOULD employer des IDs explicites.

Les identifiants internes d'un noyau, offsets de fichier et numéros de face sont interdits comme IDs persistants.

8. Expressions

La série 0.1 offre :

- littéraux numériques, chaînes, booléens et null ;
- quantités avec unité ;
- références qualifiées ;
- listes ;
- appels à arguments positionnels ou nommés ;
- opérateurs `+`, `-`, `*`, `/`, `^` ;
- comparaisons ;
- `and`, `or`, `not`.

Les appels nommés utilisent = :

```
rectangle(width, depth, centered = true)
```

L'absence de boucle générale garantit la terminaison syntaxique. Les motifs finis sont des opérations géométriques, pas des boucles du langage.

9. Attributs, provenance et incertitude

Les attributs sont une liste clé-valeur située après une déclaration. Les clés standard incluent `id`, `profile`, `status`, `evidence`, `min`, `max`, `step_mapping` et `deprecated`.

Les états de connaissance sont :

- `observed` OU `certain` ;
- `hypothesis` ;
- `ambiguous` ;
- `unknown`.

`hypothesis`, `ambiguous` et `inferred` exigent `evidence`. Une hypothèse ne peut alimenter une sortie automatiquement acceptée avant une décision ou une règle vérifiable.

10. Profils

Profil	Contenu minimal
P1	Pièce prismatique/tournée, paramètres, sketch, extrude, cut, revolve, boolean, hole, pattern, mirror, fillet et chamfer constants
P2	P1 + matériau, datums, PMI/GD&T borné, thread et propriétés de validation
P3	P2 + occurrences, assemblage, mates, configurations et cinématique simple
P4	P3 + sweep, loft, blend, shell, draft, fillets variables et géométrie avancée

Une opération hors profil est une erreur. Un backend peut déclarer `unsupported`; il ne peut pas produire un résultat approximatif non déclaré.

11. Extensions

Une extension comprend URI, version, digest, licence, opérations, types, rôles topologiques, mapping STEP, règles de fallback et tests. Une extension inconnue peut être parsée et conservée. Elle ne peut pas être exécutée.

Les extensions ne contiennent pas de code natif arbitraire. Les implémentations sont fournies par plugins signés ou workers isolés. Un dialecte MLIR peut être généré en interne, mais son assembleur n'est pas l'encodage d'archive.

12. Compilation

Ordre logique :

1. UTF-8 et normalisation ;
2. lexing et parsing sans récupération silencieuse ;
3. versions et extensions ;
4. symboles et IDs ;
5. types et unités ;
6. DAG ;
7. capacités du backend ;
8. solveur ;
9. features ;
10. lignée et références ;
11. géométrie, topologie et PMI ;
12. propriétés et scénarios ;
13. export et registre des pertes ;
14. commit atomique.

13. Graphe canonique JSON 0.1

Le schéma est publié dans `schemas/morphoia-ir-0.1.schema.json`. Le JSON comprend format, version, namespace, modèles, IDs, expressions, ordre du DAG, contrat d'exécution et hash SHA-256.

L'encodage JSON est destiné au débogage, aux tests et à l'échange entre composants du POC. Une future sérialisation STEP Part 21 ou Part 26 doit réutiliser la sémantique ISO et déclarer les extensions.

14. Erreurs minimales

Code	Signification
MORPH-E000	Erreur lexicale
MORPH-E010	Erreur syntaxique
MORPH-E100	Profil inconnu
MORPH-E101	Déclaration dupliquée
MORPH-E102	Référence inconnue
MORPH-E103	Cycle dans le DAG
MORPH-E110	Type ou unité incompatible
MORPH-E120	Opération hors profil
MORPH-E130..134	Référence persistante incomplète ou ambiguë par contrat
MORPH-E140	Catégorie de tolérance invalide
MORPH-E150	Hypothèse sans preuve

Les diagnostics doivent être stables, localisés et disponibles en JSON.

15. Sécurité et limites

Le parser doit imposer des limites de taille, profondeur, nombre de noeuds et complexité des expressions dans le profil de production. Les imports distants ne sont jamais résolus sans allowlist et digest. Les plugins et parseurs externes sont isolés.

16. Critères de maintien de la façade

La syntaxe ne sera maintenue vers 1.0 que si les benchmarks démontrent :

- bijectivité source-graphe-source sur 100 % du profil ;
- hashes identiques pour deux parseurs indépendants ;
- validité syntaxique IA cible au moins 98 % ;
- gain préenregistré de validité sémantique face à CadQuery ;
- réduction préenregistrée du temps de correction humaine ;
- sécurité, terminaison et limites de ressources ;
- portabilité P1/P2 sur deux backends.

En cas d'échec, le graphe canonique et l'API restent ; la façade est abandonnée.

Grammaire EBNF complète

```
(* MORPHOIA experimental textual facade 0.1. *)

CompilationUnit = Header, [ NamespaceDecl ], { ImportDecl }, ModelDecl, { ModelDecl }, EOF ;

Header          = "morphoia", Version, ";" ;
Version         = Number | String | Identifier ;
NamespaceDecl   = "namespace", QualifiedName, ";" ;
ImportDecl      = "import", String, [ "as", Identifier ], ";" ;

ModelDecl       = "model", Identifier, [ Attributes ], Block ;
Block           = "{", { Node }, "}" ;

Node            = Assignment
                | SimpleDeclaration
                | SketchDeclaration
                | FeatureDeclaration
                | AnonymousBlock
                | NamedBlock
                | NamedValueStatement
                | ExpressionStatement
                | SetStatement
                | ExtensionStatement ;

Assignment      = QualifiedName, "=", Expression, ";" ;

SimpleDeclaration = SimpleKind, Identifier,
                    [ ":", TypeRef ],
                    [ "=", Expression ],
                    [ Attributes ],
                    ( ";" | Block ) ;

SimpleKind      = "parameter" | "constant" | "datum" | "reference"
                | "material" | "pmi" | "tolerance" ;

SketchDeclaration = "sketch", Identifier, "on", Expression,
                    [ Attributes ], Block ;

FeatureDeclaration = "feature", Identifier, ":", QualifiedName,
                    [ Attributes ], Block ;

AnonymousBlock  = AnonymousBlockKind, [ Attributes ], Block ;
AnonymousBlockKind = "units" | "metadata" | "validate" ;

NamedBlock      = NamedBlockKind, Identifier, [ Attributes ], Block ;
NamedBlockKind  = "assembly" | "scenario" | "extension" ;

NamedValueStatement = NamedValueKind, Identifier,
                    [ ":", TypeRef ], "=", Expression,
                    [ Attributes ], ";" ;
NamedValueKind    = "profile" | "dimension" | "let" | "output" ;

ExpressionStatement = ExpressionStatementKind, Expression,
                    [ Attributes ], ";" ;
ExpressionStatementKind = "constraint" | "assert" | "expect" ;

SetStatement     = "set", QualifiedName, "=", Expression, ";" ;

ExtensionStatement = Identifier, ( Identifier, Block | Expression, [ Attributes ], ";" ) ;

Attributes      = "[", [ Attribute, { ",", Attribute } ], "]" ;
Attribute       = Identifier, "=", Expression ;

Expression      = OrExpression ;
OrExpression     = AndExpression, { ( "or" | "||" ), AndExpression } ;
AndExpression    = EqualityExpression, { ( "and" | "&&" ), EqualityExpression } ;
EqualityExpression = RelationalExpression,
                    { ( "==" | "!=" | "~=" ), RelationalExpression } ;
RelationalExpression = AddExpression,
                    { ( "<" | "<=" | ">" | ">=" ), AddExpression } ;
AddExpression    = MultiplyExpression, { ( "+" | "-" ), MultiplyExpression } ;
MultiplyExpression = PowerExpression, { ( "*" | "/" ), PowerExpression } ;
PowerExpression  = UnaryExpression, [ "^", PowerExpression ] ;
UnaryExpression  = [ "+" | "-" | "not" ], PrimaryExpression ;
```

```

PrimaryExpression
    = Literal
    | Call
    | QualifiedName
    | List
    | "(" , Expression , ")" ;

Call
    = QualifiedName , "(" , [ Arguments ] , ")" ;
Arguments
    = Argument , { "," , Argument } ;
Argument
    = [ Identifier , "=" ] , Expression ;

List
    = "[" , [ Expression , { "," , Expression } ] , "]" ;
Literal
    = Quantity | Number | String | Boolean | "null" ;
Quantity
    = Number , UnitSymbol ;
Boolean
    = "true" | "false" ;

TypeRef
    = QualifiedName ;
QualifiedName
    = Identifier , { "." , Identifier } ;
UnitSymbol
    = Identifier ;

Identifier
    = ASCII_Letter_Or_Underscore ,
      { ASCII_Letter_Or_Underscore | Digit } ;
Number
    = Digit , { Digit } ,
      [ "." , Digit , { Digit } ] ,
      [ ( "e" | "E" ) , [ "+" | "-" ] , Digit , { Digit } ] ;
String
    = JSON_String ;

ASCII_Letter_Or_Underscore
    = "A".. "Z" | "a".. "z" | "_" ;
Digit
    = "0".. "9" ;
EOF
    = ? end of input ? ;

(* Lexical notes:
  - Encoding is UTF-8. Identifiers are ASCII; international labels use String.
  - Comments are // to end of line or non-nested /* ... */.
  - Strings use JSON escapes.
  - NaN, Infinity and hexadecimal numbers are forbidden.
  - A Number followed by an identifier is a Quantity only when the active
    unit registry recognizes the identifier.
*)

```

Catalogue d'opérations candidate

1. Règles communes

Chaque opération possède :

- un nom qualifié et une version ;
- un profil minimal ;
- des arguments typés et nommés ;
- des préconditions ;
- des résultats immuables ;
- des rôles topologiques de sortie ;
- des cas d'échec explicites ;
- un mapping STEP conceptuel ;
- un fallback et un registre des pertes ;
- des scénarios d'édition obligatoires.

Aucun défaut influençant la géométrie ne dépend silencieusement du backend. Les noms exacts des entités EXPRESS ne seront figés qu'après analyse des schémas officiels SMRL.

2. Esquisses, contraintes et dimensions

Sketch

Champ	Contrat
Entrées	Plan ou frame2 explicite, unités, entités, contraintes, politique solveur
Sorties	Esquisse, régions fermées, état de contrainte, diagnostics
Profil	P1
Invariants	Repère explicite, aucune Workplane implicite, branches du solveur enregistrées
Réemploi	ISO 10303-112, FreeCAD Sketcher ou SolveSpace

Constraint

Le profil initial couvre coïncidence, horizontalité, verticalité, parallélisme, perpendicularité, tangence, concentricité, égalité, symétrie, distance, angle, rayon, diamètre, point-sur-courbe et fixité explicite.

Les contraintes industrielles sont dures. Les poids probabilistes appartiennent au générateur IA et ne sont pas conservés comme règles autoritatives.

Dimension

Une dimension relie une cible, une quantité typée, un mode `driving` ou `reference`, une provenance et, en P2, une tolérance dimensionnelle. Une dimension explicitement observée doit être exacte ou provoquer abstention.

3. Features volumiques P1

Extrude

Champ	Contrat
Entrées	Région2, direction, limite initiale, limite finale, dépouille explicite
Sorties	Solide ou surface, rôles <code>cap.start</code> , <code>cap.end</code> , <code>lateral</code>
Echecs	Profil ouvert, auto-intersection, direction nulle, résultat non-manifold
Edition	Distance, sens, symétrie, région et dépouille
Mapping	ISO 10303-111/-113, géométrie résultat ISO 10303-42

Cut

cut est un booléen soustractif spécialisé qui conserve l'intention de poche ou d'enlèvement. Il exige cible, outil, politique d'intersection et comportement si l'outil n'intersecte pas la cible.

Revolve

Entrées : région, axe, angle, sens, limite et politique de fermeture. Les cas où le profil croise l'axe, crée une auto-intersection ou produit une feuille nulle doivent être définis par le profil et non par un défaut backend.

Boolean

Opérations union, subtract et intersect. La cible primaire et les outils sont ordonnés, car la lignée topologique et la robustesse numérique peuvent dépendre de l'ordre même si la théorie ensembliste paraît commutative.

Le backend rapporte : succès exact, succès avec healing déclaré, résultat vide, non-manifold, tolérance dépassée ou divergence.

Hole

Paramètre	Valeurs minimales
Support	Plan, face ou datum résolu de manière unique
Placement	Point2 ou motif de points dans un repère explicite
Axe	Direction ou référence d'axe
Forme	Simple, counterbore, countersink, spotface
Terminaison	Blind, through_all, up_to_face, up_to_next
Filetage	None, cosmetic, semantic ou modeled
Sorties	hole.wall, hole.bottom, entry, axe sémantique

Pattern

Le motif réexécute ou transforme une seed selon une politique explicite. Les variantes linéaire, circulaire et table finie sont P1. Chaque instance possède une clé stable ; l'identité ne dépend pas seulement de sa position dans une liste.

Le comportement en cas de collision, suppression d'une instance, fusion de résultats et références externes doit être déclaré.

Mirror

Entrées : seed, plan ou frame, politique de copie ou de fusion. Le miroir doit déclarer le traitement de la chiralité, des repères locaux et des PMI associés.

Fillet

P1 couvre rayon constant et continuité G1. Les arêtes sont des références persistantes. Les débordements, coins, tangences, rayons impossibles et propagation tangentielle sont explicites. Les lois variables relèvent de P4.

Chamfer

Modes : deux distances, distance-angle ou symétrique. La face de référence, le sens et le comportement aux sommets sont explicites.

4. Géométrie avancée P4

Sweep

Entrées : profil, chemin, frame d'orientation, lois d'échelle/torsion, continuité, coins et politique d'auto-intersection. Les variantes Frenet, corrected Frenet et direction fixe ne sont pas interchangeables.

Loft

Entrées : sections ordonnées, guides, correspondance des sommets, continuité et conditions aux extrémités. La résolution automatique de correspondance doit être enregistrée comme décision ou refusée.

Blend

blend désigne une surface de transition entre frontières ou surfaces avec continuité G0/G1/G2. Il ne doit pas être confondu avec la feature d'arrondi d'arêtes fillet.

Shell

Entrées : corps, faces retirées, épaisseur, côté, stratégie de coins. Les changements topologiques, auto-intersections et zones d'épaisseur nulle sont des erreurs ou pertes déclarées.

Draft

Entrées : faces, direction de tirage, élément neutre, angle et traitement des faces tangentes. Un draft automatique propre à une CAO n'est pas portable sans contrat supplémentaire.

5. Références et construction

Datum

Datums de point, axe, plan et cible. En P2, la sémantique GD&T distingue le datum théorique, le datum feature et le datum simulé.

Coordinate System

Origine, axes, chiralité et unités. Les axes doivent être orthogonaux et normalisés dans la tolérance noyau. La forme canonique n'utilise pas d'angles d'Euler ambigus.

Reference Plane

Créé à partir d'un datum, d'un frame, d'une face plane ou d'un offset explicite. L'orientation et le sens normal sont obligatoires.

Reference Axis

Créé à partir d'une ligne, d'une face cylindrique, de deux points ou d'un repère. La référence source reste persistante et doit être unique.

6. P2 - matériau, tolérances et PMI

Material

Le matériau référence désignation, norme, version, propriétés, température, source et incertitude. MORPHOIA ne crée pas une nouvelle base matériaux ; il relie des identifiants externes et conserve les valeurs nécessaires à la validation.

Thread

Le filetage déclare standard, désignation, diamètre, pas, classe, sens, longueur et représentation semantic, cosmetic ou modeled. Un export qui remplace un filetage modélisé par une annotation produit une perte explicite.

Tolerance

kernel_tolerance, measurement_uncertainty, dimensional_tolerance et gdt_zone sont quatre types distincts. La cible, l'unité, la distribution ou zone et la provenance sont obligatoires selon le type.

PMI

Le PMI porte sémantique et association exacte. P2 couvre dimensions, datums, tolérances de forme/orientation/localisation bornées, état de surface et notes structurées. La présentation graphique est optionnelle et dérivée.

Le glyphe seul n'est jamais considéré comme tolérance sémantique.

7. P3 - Assembly

Un assemblage distingue :

- définition produit ;
- occurrence ;
- transformation rigide ;
- mate/contrainte ;
- configuration ;
- substitution ;
- BOM ;
- enveloppe et interférence.

Le profil initial couvre occurrences rigides, coïncidence, concentricité, distance, angle, fixité et cinématique simple. Flexibilité, déformation, câblage et grands mécanismes nécessitent des extensions ultérieures.

8. Mécanisme d'ajout d'opérations

Une opération d'extension doit fournir :

1. URI, version et digest ;
2. types d'entrée et de sortie ;
3. préconditions et postconditions ;
4. rôles topologiques ;
5. déterminisme et limites ;
6. mapping STEP ou écart ;
7. fallback B-rep ;
8. registre des pertes par backend ;
9. cas dégénérés ;
10. scénarios d'édition ;
11. suite golden et adversariale ;
12. Component Justification Record.

Une opération inconnue peut être préservée dans le graphe. Elle ne peut pas être exécutée sans implémentation et manifest validés.

9. Matrice de réemploi

Domaine	Réemploi obligatoire ou prioritaire
Procédures	ISO 10303-55
Paramètres/contraintes	ISO 10303-108
Assemblages	ISO 10303-109 et AP242
Solides procéduraux	ISO 10303-111
Esquisses	ISO 10303-112
Features mécaniques	ISO 10303-113
Géométrie et topologie	ISO 10303-42
Produit, configuration, PMI	AP242
Noyau	OCCT, backends commerciaux optionnels
Solveur	FreeCAD Sketcher, SolveSpace ou D-Cubed

Cette matrice interdit de déclarer une opération nouvelle simplement parce qu'une API existante est moins agréable. L'écart sémantique doit être démontré.

SDK et API MORPHOIA

1. Statut et portée

Ce document décrit à la fois l'API réellement implémentée dans le prototype Python et l'architecture cible du SDK. Ces deux états ne doivent pas être confondus.

MORPHOIA reste un candidat expérimental 0.1. Il ne s'agit ni d'un SDK stable, ni d'une norme, ni d'une promesse de compatibilité binaire ou fonctionnelle. La façade textuelle et l'encodage JSON peuvent encore être remplacés si les benchmarks ne démontrent pas leur utilité.

Le module Python annonce actuellement 0.2.0-dev et la métadonnée de distribution 0.2.0.dev0. Ces versions de développement du paquet ne changent pas le statut des contrats de données : la façade source, le graphe canonique, les manifests et les registres décrits ici restent des formats candidats 0.1.

Légende de maturité

Marqueur	Signification
Implémenté 0.1	Code présent et couvert au moins par un test local
Partiel 0.1	Prototype présent, mais contrat ou couverture incomplet
Cible	Architecture prévue, sans implémentation utilisable dans le dépôt
Hors périmètre 0.1	Ne doit pas être attendu du prototype actuel

2. Principes imposés par la Phase 1

1. STEP/AP242 et ISO 10303-42/-55/-108/-109/-111/-112/-113 restent les ancres sémantiques ; le SDK ne crée pas une ontologie mécanique concurrente.
2. OCCT doit être réutilisé comme premier noyau exact ouvert. MORPHOIA ne réécrit pas un noyau B-rep.
3. Le modèle canonique ne contient aucun type, pointeur ou index natif d'un noyau ou d'une CAO.
4. L'IA propose des candidats. Le runtime, les solveurs, les noyaux et les validateurs décident si un candidat peut être accepté.
5. Toute approximation, réparation, substitution, omission ou incompatibilité produit une perte machine-readable. Il n'existe pas de fallback silencieux.
6. Le résultat explicite, les propriétés de validation, l'environnement et le graphe procédural doivent pouvoir être conservés ensemble.

3. Inventaire exact du prototype Python

Capacité	État	Module ou commande	Limite explicite
Lexer et parser de la façade .morph	Implémenté 0.1	<code>morphoia.parser.parse</code>	Façade expérimentale, non standardisée
Validation syntaxique et sémantique	Partiel 0.1	<code>validate_source</code> , <code>validate_file</code>	Ne valide ni B-rep, ni STEP, ni PMI exécuté
Compilation vers graphe JSON canonique	Implémenté 0.1	<code>compile_document</code>	Produit un dictionnaire ; aucun noyau n'est appelé
JSON canonique et hash sémantique	Implémenté 0.1	<code>canonical_json</code> , <code>semantic_hash</code>	Déterminisme testé seulement dans le prototype Python
Résolution topologique à trois états	Partiel 0.1	<code>morphoia.topology</code>	Filtrage exact sur candidats fournis ; ne résout pas le problème universel de nommage
Contrat Python abstrait de backend	Partiel 0.1	<code>morphoia.backends.base</code>	Pas de découverte, manifest, isolation ou négociation automatique
Backend de sérialisation JSON	Implémenté 0.1	<code>CanonicalJsonBackend</code>	Encodeur uniquement ; aucune géométrie exacte
Révisions et transactions en mémoire	Partiel 0.1	<code>morphoia.runtime.RuntimeStore</code>	Aucun stockage persistant, backend ou verrouillage multi-processus
Preuves et décisions	Partiel 0.1	<code>morphoia.provenance</code>	Primitives isolées ; pas encore reliées automatiquement au graphe
Registre des pertes Python	Partiel 0.1	<code>morphoia.losses</code>	Création manuelle ; aucun adaptateur ne le produit automatiquement
CLI <code>validate</code>	Implémenté 0.1	<code>python -m morphoia validate</code>	Diagnostics syntaxiques/sémantiques seulement
CLI <code>compile</code>	Implémenté 0.1	<code>python -m morphoia compile</code>	Écrit le graphe JSON ; aucun choix de backend
Schéma JSON du graphe	Partiel 0.1	<code>schemas/morphoia-ir-0.1.schema.json</code>	Le prototype ne lance pas automatiquement un validateur JSON Schema
Schéma du registre des pertes	Partiel 0.1	<code>morphoia-loss-register-0.1.schema.json</code>	Reflète <code>LossRegister.to_dict()</code> ; validation non automatique
Manifest backend	Contrat candidat 0.1	<code>morphoia-backend-manifest-0.1.schema.json</code>	Aucun chargeur ou négociateur n'est encore implémenté
Backend OCCT, STEP ou FreeCAD	Cible	-	Absent du dépôt actuel
Backend Parasolid ou CAO commerciale	Cible optionnelle	-	Nécessite SDK, contrat et licence
Coeur C++, ABI C et bindings	Cible	-	Aucun code C/C++ présent
Service réseau, sandbox et plugins signés	Cible	-	Aucun daemon ni mécanisme de sécurité correspondant

La déclaration des profils P1 à P4 par le backend JSON signifie qu'il peut préserver les noeuds JSON correspondants. Elle ne prouve pas qu'il sait exécuter leur géométrie ou qu'il est conforme à ces profils.

4. API Python réellement disponible

Les seuls symboles exportés à la racine du paquet sont :

```
from morphoia import (
    canonical_json,
    compile_document,
    parse,
    semantic_hash,
    validate_file,
    validate_source,
    RuntimeStore,
)
```

4.1 Validation sûre

```
from morphoia import compile_document, validate_source

source = """morphoia 0.1;
model Block [profile = "P1"] {
  parameter height: length = 10 mm;
}
"""

result = validate_source(source)
if not result.ok or result.document is None:
    for diagnostic in result.diagnostics:
        print(diagnostic.code, diagnostic.severity.value, diagnostic.message)
else:
    ir = compile_document(result.document)
    print(ir["semantic_sha256"])
```

`validate_source(source: str) -> ValidationResult` capture les erreurs du lexer et du parser, puis applique les règles sémantiques disponibles. `validate_file(path) -> ValidationResult` lit un fichier UTF-8 et appelle la même pipeline. Les erreurs d'accès au fichier ne sont pas transformées en diagnostics MORPHOIA.

`parse(source: str) -> Document` est une API de plus bas niveau. Elle peut lever `LexError` ou `ParseError` et ne lance pas la validation sémantique.

4.2 Compilation candidate

`compile_document(document) -> dict[str, Any]` abaisse un AST en graphe JSON déterministe et ajoute `semantic_sha256`. Le contrat d'appel impose de valider le document avant compilation ; la fonction ne répète pas elle-même toutes les validations.

`canonical_json(ir, indent=2) -> str` trie les clés et termine par une nouvelle ligne. `semantic_hash(ir) -> str` calcule SHA-256 sur l'encodage compact trié en ignorant le champ `semantic_sha256` déjà présent.

Ces fonctions ne prouvent ni la reproductibilité multi-langage, ni l'équivalence de deux B-rep, ni le déterminisme d'un noyau.

4.3 AST et diagnostics

`morphoia.model` contient des dataclasses immuables pour les positions, expressions, noeuds, modèles et documents. `ValidationResult.ok` est vrai si un document existe et qu'aucun diagnostic de sévérité `error` n'est présent.

Ces classes sont nécessaires au prototype, mais elles ne sont pas exportées à la racine et ne bénéficient d'aucune garantie de stabilité 0.x.

4.4 Références topologiques

`resolve_reference(query, candidates)` filtre les candidats sur :

- type d'entité ;
- feature productrice ;
- rôle sémantique ;
- termes exacts de signature ;
- contexte d'adjacence demandé.

Le résultat est unique, ambiguous ou missing. Accéder à `.entity` sur un résultat non unique lève `LookupError`. Le score flou, la propagation de lignée au travers d'opérations OCCT et la résolution après édition sont des cibles, pas des capacités actuelles.

4.5 Backend Python minimal

Le contrat actuel est volontairement petit :

```
class Backend(ABC):
    def capabilities(self) -> BackendCapabilities: ...
    def execute(self, canonical_ir: dict[str, Any]) -> BackendResult: ...
```

`BackendCapabilities` contient actuellement `name`, `version`, `profiles`, `operations`, `exact_brep`, `pmi`, `deterministic_mode` et `license`. `BackendResult` contient `success`, `artifact_uri`, `validation_properties`, `losses` et `diagnostics`.

CanonicalJsonBackend écrit un graphe JSON et retourne son hash comme propriété de validation. Il ne valide pas le graphe contre son JSON Schema et ne produit pas encore un registre de pertes conforme au schéma candidat.

4.6 Transactions, provenance et pertes

RuntimeStore fournit un historique immuable en mémoire. Une transaction travaille sur une copie du graphe, peut recevoir des fonctions de validation, effectue un commit atomique ou un rollback, détecte une révision de base obsolète et permet undo/redo. Cette preuve de contrat ne fournit pas de persistance, d'isolation entre processus, de solveur ou d'exécution géométrique.

Evidence vérifie notamment la forme d'un SHA-256. Decision représente candidate, accepted, rejected ou abstained, borne la confiance à [0, 1] et exige preuve et acteur pour une décision acceptée. Ces objets ne sont pas encore sérialisés dans l'IR ni résolus par le runtime.

LossRecord et LossRegister fournissent un registre simple sérialisable en dictionnaire. Une perte error ou fatal fait passer blocks_commit à vrai. Le schéma 0.1 reflète cette représentation. Les IDs de transformation, environnements verrouillés, liens de preuves et compteurs détaillés décrits plus loin restent une cible d'enrichissement.

5. CLI disponible

```
PYTHONPATH=src python -m morphoia validate model.morph
PYTHONPATH=src python -m morphoia validate model.morph --json
PYTHONPATH=src python -m morphoia compile model.morph -o model.mcir.json
```

Codes de sortie : 0 si la validation réussit, 1 si elle échoue. La commande compile valide d'abord la source, puis écrit le JSON. Il n'existe actuellement aucune commande execute, export-step, diff, bench, serve ou backend list.

6. Architecture cible du SDK

Les sections suivantes décrivent une cible et non du code livré.

6.1 Coeur C++ et ABI de plugins

Le coeur déterministe cible C++ et doit porter :

- modèle canonique typé ;
- unités et quatre catégories de tolérances distinctes ;
- transactions, DAG, commit et rollback ;
- solveur et résolution des références ;
- exécution de backend ;
- propriétés de validation et rapports structurés ;
- limites de ressources et journalisation.

Les plugins doivent utiliser une ABI C versionnée avec handles opaques. Une API C++ ergonomique peut l'envelopper, mais elle ne constitue pas l'ABI stable. Aucun type OCCT, Parasolid, FreeCAD ou CAO native ne traverse cette frontière.

6.2 Manifest backend

Tout backend cible fournit un document conforme à

[morphoia-backend-manifest-0.1.schema.json](../../schemas/morphoia-backend-manifest-0.1.schema.json). Le manifest déclare notamment :

- identité, version et état expérimental ;
- mode d'entrée : bibliothèque, worker ou service ;
- profils, opérations et formats réellement supportés ;
- B-rep exacte, PMI, assemblages et modes déterministes ;
- plateformes, GPU, isolation et permissions ;
- licence, redistribution, cloud et restrictions ;

- état de qualification et lien vers le rapport de test.

Le chargement cible suit : découverte, validation du manifest, contrôle de licence et d'intégrité, négociation de capacités, préflight, exécution isolée, validation, collecte du registre des pertes, puis destruction du contexte.

6.3 API cible par responsabilité

Domaine	Responsabilité cible
model	documents, révisions, noeuds, expressions et IDs canoniques
profiles	P1-P4, opérations, capacités et règles de fallback
execution	planification, transactions, solveur et backend
topology	lignée, requêtes, ambiguïtés et décisions humaines
validation	propriétés, scénarios, diff et rapports
losses	production, fusion et validation du registre des pertes
io	STEP/AP242, QIF, AP238 et vues dérivées
provenance	sources, preuves, licences, décisions et confiance
ml	candidats IA non autoritatifs
bench	corpus, environnements et résultats reproductibles

6.4 Backends cibles

Backend	Mode cible	Rôle autorisé
OCCT/OCAF/XDE	bibliothèque ou worker	backend exact ouvert initial, STEP et propriétés
FreeCAD	worker headless versionné	oracle de recompute, Sketcher et cible ouverte
Parasolid/ACIS/CGM/C3D	plugin sous licence	backend ou oracle optionnel
CAO native	worker par produit/version	import/export borné et cible utilisateur
Blender	worker isolé	rendu et données synthétiques uniquement
glTF/OpenUSD/JT	encodeur de vue	représentation dérivée, jamais autorisé

6.5 Services cibles

Un service réseau éventuel enveloppe les mêmes contrats et ne remplace pas le SDK local. Les appels longs utilisent des jobs idempotents avec artefacts adressés par contenu. Un résultat doit inclure versions, environnement, diagnostics, propriétés et registre des pertes.

Le transport cible peut être gRPC ou HTTP, mais aucune route ou compatibilité réseau n'est promise en 0.1.

7. Erreurs, pertes et diagnostics

Le prototype utilise des diagnostics MORPH-E... structurés pour la source, mais les diagnostics de backend sont encore de simples chaînes.

La cible impose :

- un code stable et namespacé ;
- une sévérité ;
- une localisation source ou un ID canonique ;
- le backend, sa version et l'opération ;
- une preuve ou propriété mesurée ;
- une action recommandée ;
- un lien vers une perte lorsque le résultat est dégradé.

Le prototype peut construire manuellement un document conforme à

[morphoia-loss-register-0.1.schema.json](../schemas/morphoia-loss-register-0.1.schema.json). Il ne branche toutefois pas automatiquement ce registre sur les backends ou les transactions et ne le valide pas automatiquement. La cible exige qu'une transformation non lossless produise ce registre sans intervention du caller, puis l'enrichisse dans une version ultérieure avec environnement, preuves et références canoniques.

8. Versioning et compatibilité

- Les versions du paquet, de la façade, de l'IR, des profils, des schémas et de chaque backend sont indépendantes.
- Toute révision 0.x peut rompre la compatibilité avec justification et note de migration.
- Un backend n'est jamais déclaré « compatible MORPHOIA » sans profil, version, plateforme et rapport de test.
- Une extension inconnue peut être préservée, mais elle n'est pas exécutée.
- Les mots stable, certified, standard et 1.0 sont interdits avant les gates définis dans le cahier des charges.

9. Sécurité cible

- aucune exécution de Python arbitraire dans le chemin de confiance ;
- parseurs et adaptateurs externes hors processus ;
- quotas CPU, GPU, mémoire, taille d'entrée et temps ;
- réseau et système de fichiers refusés par défaut ;
- manifests, plugins, modèles et artefacts signés après G1 ;
- aucune donnée client réutilisée pour l'entraînement sans droit explicite ;
- logs sans secrets et provenance de chaque artefact.

Ces protections sont non implémentées dans le prototype Python actuel.

10. Gates d'implémentation SDK

Gate	Preuve minimale
SDK-0	APIs Python actuelles, tests parser/topologie et schémas candidats
SDK-1	Validation JSON Schema automatique et registres de pertes produits
SDK-2	Backend OCCT P1, résultat B-rep et propriétés de validation
SDK-3	Second backend indépendant, manifeste et tests différentiels
SDK-4	ABI C, bindings Python, sandbox et matrice multi-plateforme
SDK-5	Deux implémentations indépendantes et suite de conformité publique

Le dépôt actuel se situe à SDK-0 partiel.

Validation et conformité MORPHOIA

1. Statut

MORPHOIA est un candidat expérimental 0.1. Le dépôt fournit quelques validateurs exécutables, mais aucune suite de conformité industrielle, aucun backend géométrique qualifié et aucune certification.

Dans ce document :

- validation signifie qu'un contrôle déterminé a été exécuté ;
- conformité candidate signifie qu'un profil, une version, un backend, une plateforme et un corpus ont tous été explicitement identifiés ;
- certification est réservée à une gouvernance et un organisme qui n'existent pas encore.

Le succès du prototype Python ne démontre ni compatibilité STEP, ni validité B-rep, ni portabilité multi-CAO.

2. Contrôles réellement implémentés

Contrôle	État 0.1	Preuve dans le dépôt	Ce qu'il ne prouve pas
Erreurs lexicales et syntaxiques	Implémenté	lexer.py, parser.py, validator.py	Sémantique géométrique
Version de façade 0.1	Implémenté	MORPH-E001	Compatibilité future
Doublons de déclarations	Implémenté	MORPH-E101	Unicité globale multi-document
Références inconnues	Implémenté	MORPH-E102	Résolution dans une B-rep réelle
Cycles du graphe de construction	Implémenté	MORPH-E103	Cycles algébriques du solveur
Cohérence dimensionnelle de paramètres	Partiel	MORPH-E110	Analyse complète de toutes les expressions et PMI
Disponibilité des opérations par profil	Partiel	MORPH-E120	Exécution réelle de P1-P4
Forme minimale des références persistantes	Partiel	MORPH-E130 à E134	Nommage persistant après opérations de noyau
Quatre catégories de tolérance	Partiel	MORPH-E140	Calcul ou vérification métrologique
Provenance des hypothèses	Partiel	MORPH-E150	Authenticité ou qualité de la preuve
Résolution unique/ambiguous/missing	Implémenté sur candidats fournis	topology.py, tests unitaires	Recherche, lignée ou score sur B-rep
Hash sémantique déterministe	Implémenté en Python	compiler.py, test de répétition	Hash multi-langage ou équivalence géométrique
Commit/rollback et undo/redo atomiques	Implémenté en mémoire	runtime.py, tests unitaires	Persistence, concurrence distribuée ou géométrie
Conflit de révision optimiste	Implémenté en mémoire	RevisionConflict	Verrouillage multi-processus
Preuves et décisions acceptées	Partiel	provenance.py	Intégration automatique au graphe et vérification de la source
Schéma JSON de l'IR	Contrat candidat	morphoia-ir-0.1.schema.json	Le compilateur ne le valide pas automatiquement
Registre des pertes	Partiel	losses.py et schéma 0.1	Construction manuelle ; aucun backend ne le produit automatiquement
Manifest backend	Schéma candidat uniquement	morphoia-backend-manifest-0.1.schema.json	Aucun plugin n'est découvert ou qualifié

Il n'existe actuellement aucun contrôle de solide, de surface, de topologie OCCT, de healing, de propriétés de masse, de PMI, de STEP round-trip, de scénario d'édition ou de comparaison multi-backend.

3. Niveaux de validation cibles

Niveau	Objet	État actuel
V0 - Source	lexer, parser, types, unités, symboles et DAG	Partiel
V1 - Graphe canonique	JSON Schema, invariants, profils, provenance et hash	Partiel
V2 - Préflight backend	manifest, capacités, licence, environnement et quotas	Non implémenté
V3 - Géométrie	B-rep, manifold, tolérances, propriétés et healing	Non implémenté
V4 - Comportement	scénarios d'édition, lignée et références	Non implémenté
V5 - Interopérabilité	export/import, STEP/AP242, PMI et pertes	Non implémenté
V6 - Différentiel	mêmes cas sur deux noyaux ou CAO	Non implémenté
V7 - Conformité	corpus public, implémentation indépendante et gouvernance	Non atteint

Un niveau supérieur n'annule jamais les résultats inférieurs. Un modèle peut être syntaxiquement valide et géométriquement invalide, ou géométriquement valide et comportementalement non conforme.

4. Modèle de verdict

Chaque contrôle cible retourne l'un des verdicts suivants :

Verdict	Sens
pass	exigence satisfaite avec preuve
fail	exigence obligatoire violée
warning	résultat utilisable selon politique, perte déclarée
skipped	contrôle prévu mais non exécuté
not_applicable	contrôle hors du profil déclaré

Un contrôle skipped ne peut pas être interprété comme pass. Un rapport de conformité doit échouer si un contrôle obligatoire du profil est skipped, si une ambiguïté est résolue silencieusement ou si une perte requise est absente.

Chaque résultat cible contient au minimum : identifiant d'exigence, profil, verdict, mesure, seuil, unité, IDs canoniques affectés, preuves, backend, environnement et diagnostics.

5. Validation du graphe

La pipeline cible V0/V1 est :

1. parser la façade candidate ou lire directement l'IR ;
2. valider le JSON Schema correspondant à la version exacte ;
3. résoudre imports, namespaces et extensions ;
4. vérifier types, dimensions et unités ;
5. vérifier unicité des IDs et intégrité des références ;
6. construire le DAG et refuser les cycles de construction ;
7. vérifier profils et capacités demandées ;
8. vérifier preuves, licences et états d'incertitude ;
9. recalculer le hash sémantique ;
10. produire un rapport structuré.

Le prototype exécute seulement une partie de ces étapes sur la façade source.

6. Validation géométrique cible

Pour chaque résultat B-rep, V3 doit contrôler :

- validité topologique selon le noyau et vérifications indépendantes possibles ;
- orientation des shells et fermeture des solides ;

- manifold/non-manifold selon le profil ;
- courbes 3D, p-curves et cohérence arête-face ;
- tolérances par entité et dépassements ;
- auto-intersections, singularités et géométries dégénérées ;
- nombre de corps, shells, faces, arêtes et sommets ;
- boîte englobante, aire, volume, centre de masse et inertie ;
- correspondance entre rôles topologiques attendus et entités produites ;
- événements de healing, avec état avant/après et perte associée.

La valeur exacte des seuils appartient au profil et au corpus. Une réparation qui modifie topologie, dimension ou association PMI n'est jamais un succès silencieux.

7. Validation comportementale cible

Un scénario d'édition contient :

- révision de départ et environnement verrouillé ;
- patch de paramètres ou de structure ;
- propriétés attendues ou invariants ;
- références qui doivent rester uniques, devenir ambiguës ou disparaître ;
- résultat attendu : commit ou rollback ;
- tolérances de comparaison ;
- pertes autorisées et interdites.

Le corpus minimal couvre variation nominale, valeurs limites, suppression, réordonnement lorsque permis, changement de motif, collision, résultat vide, référence cassée et cas dégénéré.

La réussite géométrique de l'état initial ne remplace pas ces scénarios.

8. Registre des pertes

Chaque import, exécution, migration, export ou round-trip non lossless produit un document conforme à `[morphoia-loss-register-0.1.schema.json](../schemas/morphoia-loss-register-0.1.schema.json)`.

Le format réellement produit par `LossRegister.to_dict()` contient :

- `schema = "morphoia.loss-register/0.1"` ;
- opération, backend source et backend cible ;
- `blocks_commit` ;
- une liste `records`.

Chaque record contient code, catégorie, sévérité, sujet, message, capacités source/cible, mitigation et caractère réversible. Les catégories implémentées couvrent géométrie, topologie, historique paramétrique, contraintes, PMI, matériau, assemblage, provenance, identité et comportement. Les sévérités sont `info`, `warning`, `error` et `fatal`.

`blocks_commit` doit être vrai dès qu'au moins un record est `error` ou `fatal`, et faux sinon. Le schéma 0.1 encode cette relation. La classe Python calcule également cette valeur.

Le prototype ne relie pas encore automatiquement un registre à un backend, une transaction ou un rapport de validation. Les éléments suivants restent la cible d'une version ultérieure : identifiant de transformation, environnement verrouillé, objets source/cible, preuves, fallback structuré et résumé par catégorie.

9. Qualification d'un backend

Un backend cible fournit un manifest conforme à

`[morphoia-backend-manifest-0.1.schema.json](../schemas/morphoia-backend-manifest-0.1.schema.json)`. La qualification est toujours bornée par :

backend x version x plateforme x profil x opération x format x PMI x configuration

Étapes cibles :

1. valider le manifest et l'intégrité du binaire ;
2. vérifier licence, redistribution et mode cloud ;
3. exécuter microcas et corpus golden ;
4. exécuter cas négatifs et adversariaux ;
5. répéter en mode déterministe ;
6. mesurer performance et mémoire ;
7. exécuter scénarios d'édition ;
8. comparer à un second backend lorsque requis ;
9. produire rapport, matrice de capacités et registres des pertes ;
10. signer le résultat de qualification.

En série 0.1, qualification.status est limité à untested, self-tested ou candidate. Le terme certifié n'est pas autorisé.

10. Profils candidats

Profil	Périmètre minimal	Condition de claim
P1	esquisse contrainte, extrusion, révolution, trou, booléen, motif, miroir, congé et chanfrein constants	toutes les opérations déclarées testées, pertes et fallbacks explicites
P2	P1 + matériau, datums, tolérances et PMI borné	associations PMI validées après édition et round-trip
P3	P2 + occurrences, placements, mates et configurations simples	définition/occurrence et scénarios d'assemblage validés
P4	surfaces/features avancées	profil séparé ; ne bloque pas P1/P2

Une claim candidate doit prendre la forme :

MORPHOIA candidate 0.1 / profil P1 / backend <nom>@<version> /
plateforme <système-architecture> / suite <version> / rapport <URI>

Les termes « compatible MORPHOIA », « conforme » ou « certifié » sans cette portée sont interdits.

11. Corpus et séparation des données

Corpus	Fondation	MVP	Cible 1.0 conditionnelle
Microcas noyau	1 000	10 000	50 000
Pièces P1/P2	500-2 000	10 000+	50 000+
Scénarios d'édition	5+ par pièce	50 000+	250 000+
Cas adversariaux	250	2 000	10 000
Backends/oracles	2	2 noyaux + 2 CAO	2 noyaux + 4 CAO

Les splits se font par famille, fournisseur et date. Des variantes proches ne doivent pas se retrouver à la fois dans apprentissage et test. Le jeu de qualification industrielle final reste aveugle et ses droits sont enregistrés.

12. Gates mesurables

Les seuils suivants sont des objectifs, pas des performances obtenues :

Indicateur	Gate POC/MVP
Validité B-rep sur cas supportés	au moins 99 %
Dimensions explicitement cotées	100 % ou abstention
Conservation unités/tolérances du profil	100 %
Scénarios d'édition réussis sur acceptés	au moins 95 %
Ambiguïtés résolues silencieusement	0

Indicateur	Gate POC/MVP
Round-trip AP242 + propriétés	au moins 99 %
Pièces hors profil correctement refusées	au moins 95 %
Temps humain médian économisé	au moins 40 %
Gain p95 d'un portage GPU accepté	au moins x2 sans perte de précision

G1 exige P1 et P2 sur deux backends indépendants. G2 exige corpus et suite de conformité publics ainsi qu'une implémentation indépendante. G3 exige le gain industriel sur deux pilotes. Le statut de standard candidat reste interdit avant G1, G2 et G3.

13. Performance et reproductibilité

Tout benchmark enregistre :

- modèle CPU/GPU, mémoire, OS, compilateur et drivers ;
- versions du runtime, backend, solveur et dépendances ;
- corpus, digest et conditions d'accès ;
- warm-up, nombre de répétitions et concurrence ;
- temps p50/p95, mémoire maximale et taux d'échec ;
- précision, tolérances et divergences ;
- modes déterministes et seeds ;
- intervalles de confiance lorsque pertinents.

Le déterminisme sémantique exige le même graphe et le même hash. Le déterminisme de profil exige les mêmes propriétés dans les tolérances avec le même lockfile. L'équivalence multi-backend n'exige pas les mêmes octets ou la même numérotation topologique.

14. Commandes disponibles aujourd'hui

```
PYTHONPATH=src python -m morphoia validate examples/mounting_plate.morph
PYTHONPATH=src python -m morphoia compile examples/mounting_plate.morph \
-o build/mounting_plate.mcir.json
PYTHONPATH=src python -m unittest discover -s tests -v
```

Ces commandes couvrent V0 et une fraction de V1. Elles ne doivent pas être présentées comme une suite de conformité.

15. Prochain incrément vérifiable

Le prochain incrément cohérent est limité à :

1. valider automatiquement l'IR, les manifests et les registres par JSON Schema ;
2. produire un registre de pertes pour chaque backend, y compris JSON ;
3. ajouter un backend OCCT P1 avec résultat explicite ;
4. mesurer propriétés et validité B-rep ;
5. exécuter au moins cinq scénarios d'édition par pièce ;
6. connecter FreeCAD ou un second oracle ;
7. publier la matrice des pertes et non une claim générale de compatibilité.

Interopérabilité, import et export

1. Principe

MORPHOIA ne promet pas un round-trip universel. Chaque frontière produit un Loss Register dont les états sont :

- `lossless` : sémantique du profil conservée ;
- `lossy_declared` : information transformée ou abandonnée ;
- `fallback_geometry` : procédure perdue, résultat explicite conservé ;
- `reconstructed_hypothesis` : historique inféré, non certain ;
- `unsupported` : refus explicite.

Les importeurs ne créent jamais un historique certain à partir d'une B-rep ou d'un mesh. Ils créent un corps importé et, séparément, zéro ou plusieurs graphes candidats avec preuves.

2. STEP

STEP AP203

Aspect	Analyse
Avantages	Très diffusé, configuration contrôlée, géométrie exacte, assemblages de base
Limites	PMI et MBD modernes limités, pas d'historique paramétrique portable complet
Import	Géométrie, produit et assemblage ; source marquée héritée
Export	Possible pour clients legacy, avec perte déclarée de PMI/features hors sous-ensemble
Rôle	Compatibilité descendante, jamais format MORPHOIA principal

STEP AP214

Aspect	Analyse
Avantages	Couleurs, couches, contexte automobile, large base installée
Limites	Remplacé fonctionnellement par AP242, sémantique procédurale insuffisante seule
Import/export	Comme AP203, avec conservation des présentations supportées
Rôle	Legacy automobile et migration vers AP242

STEP AP242

Aspect	Analyse
Avantages	Produit, configuration, géométrie exacte, PMI/GD&T, assemblages, validation propriétés
Limites	Ressources procédurales peu implémentées de bout en bout, performance Part 21 et profils variables
Import	Autorité mécanique neutre ; procédure seulement si réellement présente et qualifiée
Export	Cible principale, avec propriété de validation et liens vers le graphe
Rôle	Contrat d'échange et d'archivage ouvert

ISO 10303-55/-108/-109/-111/-112/-113

Ces ressources ancrent le graphe procédural, les paramètres, contraintes, assemblages, solides, esquisses et features. MORPHOIA doit produire un mapping conceptuel puis un mapping EXPRESS vérifié contre les schémas officiels. Une nouvelle entité n'est admise que pour un écart documenté.

3. Formats géométriques historiques

IGES

IGES reste utile pour courbes, surfaces et archives. Il transporte mal topologie moderne, PMI associatif, configuration et historique. L'import réalise sewing/healing déclaré et marque les incertitudes d'unité ou d'orientation. L'export est un mode legacy avec pertes importantes.

BREP

B-rep désigne une famille de représentations, pas un format universel unique. Un fichier BREP OCCT peut préserver une géométrie exacte pour le backend OCCT, mais ne porte pas l'intention complète. Il sert de snapshot ou cache adressé par hash, jamais de source unique du modèle.

Parasolid XT

Aspect	Analyse
Avantages	B-rep industrielle robuste, écosystème NX/SolidWorks/Onshape et nombreux traducteurs
Limites	Spécification et noyau propriétaires, licence OEM, pas d'historique universel
Import/export	Adaptateur sous licence ; géométrie et attributs, features seulement via API hôte dédiée
Compilateur	Backend optionnel et oracle différentiel, jamais dépendance de lecture du standard

ACIS SAT/SAB

Aspect	Analyse
Avantages	B-rep industrielle, format SAT lisible dans certaines versions, large historique
Limites	Sémantique et noyau propriétaires, variantes de version, pas d'intention complète
Import/export	SDK/licence ou support de traducteur ; registre de version et pertes
Compilateur	Backend géométrique optionnel

4. Noyaux et programmabilité ouverte

Open CASCADE Technology

OCCT est le backend ouvert de référence : B-rep, courbes/surfaces, booléens, fillets, healing, tessellation, OCAF, XDE et STEP. L'adaptateur reçoit le graphe canonique et rend handles, lignées, diagnostics et propriétés. Les classes OCCT ne traversent pas l'API normative.

La compilation inverse d'une TopoDS_Shape produit un ImportedBody et des candidats de features ; elle ne reconstitue pas un historique certain.

FreeCAD

FreeCAD sert de banc d'intégration ouvert, d'oracle de recompute et de cible utilisateur. L'export peut créer un document, des paramètres, Sketcher, PartDesign et liens vers IDs MORPHOIA. Les divergences de comportement sont enregistrées.

L'import d'un document FreeCAD peut récupérer davantage d'historique qu'un STEP, mais les objets Python, workbenches et plugins non reconnus sont isolés ou marqués unsupported.

CadQuery

CadQuery est un frontend et une cible de prototypage OCCT. Un export MORPHOIA vers CadQuery peut générer du Python lisible, mais ce code n'est pas l'autorité et peut perdre PMI, transactions et contrats de référence. Un import repose sur analyse statique limitée ou exécution sandboxée ; Python arbitraire interdit la garantie générale.

OpenSCAD

OpenSCAD offre CSG et paramétrage lisible. Export pertinent pour formes CSG simples et fabrication maker. Les fillets, B-rep exacte, PMI et sketch constraints sont limités. L'import produit un arbre CSG lorsque possible, sinon un mesh et une perte.

FeatureScript

FeatureScript constitue une référence pour unités, features, version de contexte et requêtes topologiques. La génération vers Onshape passe par ses API et reste soumise au runtime cloud. L'import de code ne peut être universel car la bibliothèque et le contexte Onshape font partie de la sémantique.

5. CAO commerciales

Fusion 360

- Avantages : historique paramétrique, API, cloud, vaste base utilisateur et données de recherche Autodesk.
- Import : API autorisée, STEP ou formats natifs via SDK ; historique selon exposition de l'API.
- Export : création de sketches/features lorsque le mapping est exact, sinon STEP/B-rep et Loss Register.
- Limite : versions, état de timeline et comportements implicites.

SolidWorks

- Avantages : CAO mécanique mature, API COM/.NET, large parc et noyau Parasolid.
- Import : document natif via API/SDK autorisé ; équations, configurations et features selon version.
- Export : macro/add-in ou fichier natif, avec tests de rebuild et propriétés.
- Limite : Windows, licences, références et features propriétaires.

CATIA

- Avantages : assemblages, surfacique, MBD, industrie aéronautique/automobile.
- Import/export : API CAA/Automation et traducteurs licenciés ; mapping par version et atelier.
- Limite : richesse propriétaire, CGM, coûts et matrice de configurations.
- Rôle : cible et oracle, jamais coeur.

Creo

- Avantages : paramétrique historique, relations, family tables, MBD.
- Import/export : Creo TOOLKIT/J-Link ou SDK de traduction, avec régénération contrôlée.
- Limite : intent manager, IDs et features dépendants de version.

Siemens NX

- Avantages : Parasolid, NX Open, PMI et manufacturing intégrés.
- Import/export : NX Open et formats JT/STEP/XT ; possibilité de créer features natives bornées.
- Limite : sémantique NX et options de model update non portables.

Autodesk Inventor

- Avantages : API riche, mécanique et paramètres, intégration Autodesk.
- Import/export : add-in/API sur versions qualifiées, STEP et SAT en fallback.
- Limite : Windows, ShapeManager/ACIS dérivé, features propriétaires.

Rhino et Grasshopper

- Avantages : NURBS, surfaces libres, openNURBS partiel et graphe computationnel Grasshopper.
- Import : 3DM, objets et, si autorisé, définition Grasshopper.
- Export : géométrie NURBS, paramètres et composants spécialisés ; PMI mécanique limité.
- Rôle : surface/design computationnel et P4, non profil P1 autoritatif.

6. DCC et formats de scène

Blender

Blender est une cible de rendu, de données synthétiques, de visualisation et d'annotation. Son modèle mesh/SubD et ses modifieurs ne remplacent pas une B-rep mécanique. L'export passe par glTF, USD ou un worker Blender isolé. Les obligations GPL sont vérifiées par architecture de processus et revue juridique.

glTF

glTF est excellent pour web, GPU et livraison légère. MORPHOIA exporte mesh, matériaux visuels, hiérarchie et IDs de liaison. Un glTF ne transporte pas l'historique paramétrique, la B-rep ni le PMI sémantique complet. L'import est une source de reconstruction.

USD et OpenUSD

OpenUSD apporte composition, layers, variants, grandes scènes et collaboration DCC. MORPHOIA peut définir un schéma dérivé reliant prims et IDs canoniques, sans représenter USD comme autorité de conception. Variants USD et configurations mécaniques ne sont pas automatiquement équivalents.

Siemens JT

JT convient aux grands assemblages, LOD, DMU, PMI et visualisation. L'export garde liens produit/occurrence et identifiants. La création ou lecture complète peut nécessiter un toolkit commercial. JT reste une vue légère et une frontière d'échange, pas l'historique de construction.

7. BIM et IFC

IFC est une norme ouverte et mature pour le BIM. Ses objets, relations, property sets et Model View Definitions inspirent gouvernance et profils. Les éléments de bâtiment et la géométrie IFC ne doivent pas être réinterprétés comme features mécaniques générales. Une extension domaine architecture peut mapper vers IFC, séparée des profils P1-P4.

8. Meshes et captures

Format	Import	Export	Limites
STL	Triangle soup avec unité souvent externe	Fabrication/additif	Pas de topologie sémantique, couleur/PMI faibles
OBJ	Mesh, groupes, UV et matériaux	DCC/visualisation	Pas d'intention mécanique
PLY	Points/mesh et attributs	Scan/recherche	Pas d'historique ni PMI
Nuage de points	Preuves et incertitude capteur	Diagnostic	Reconstruction sous-déterminée

Ces formats alimentent segmentation, primitive fitting et hypothèses. La conversion directe en historique certain est interdite.

9. Fabrication et qualité

AP238/STEP-NC et ISO 14649 portent workingsteps et fabrication. QIF porte qualité, inspection et résultats métrologiques. MORPHOIA les relie aux mêmes IDs produit, datums et caractéristiques sans dupliquer leur ontologie.

10. SDK de traduction

HOOPS Exchange, 3D InterOp, CAD Exchanger et Datakit offrent une couverture native et PMI plus large que les bibliothèques ouvertes. Ils sont évalués comme adaptateurs licenciés. Leurs objets restent derrière une ABI de plugin et leur indisponibilité ne doit pas empêcher la lecture du graphe ou du snapshot neutre.

11. Compilateurs directs

Cible	Stratégie 0.x	Autorité
Graphe JSON	Lossless pour le modèle abstrait supporté	Encodage de test
OCCT	Exécution P1 puis P2	Backend de référence
STEP AP242	Mapping + B-rep + PMI + propriétés	Echange neutre principal
FreeCAD	Document et recompute	Oracle/cible ouverte
Parasolid	Plugin sous licence	Oracle commercial optionnel
Blender	Mesh, scène et annotations	Vue dérivée
glTF	Mesh/LOD/IDs	Vue web dérivée

12. Compilateurs inverses

Source	Résultat certain	Résultat hypothétique
MORPHOIA	Graphe, IDs et profils reconnus	Extension inconnue préservée
STEP procédural qualifié	Sous-ensemble mappé	Opérations non reconnues
STEP B-rep	Produit, géométrie, PMI présents	Historique et intention
Fichier natif via API	Données exposées et versionnées	Etat implicite ou feature non mappée
Mesh/scan/images	Observations et preuves	Toutes les features et dimensions non explicites
Texte/voix	Commandes et intentions linguistiques	Paramètres absents et choix géométriques

13. Matrice de qualification

Chaque adaptateur publie :

backend x version x plateforme x profil x operation x PMI x configuration

Une cellule contient `lossless`, `lossy_declared`, `fallback_geometry`, `unsupported` et le taux de réussite du corpus. Le terme compatible est interdit sans profil et version.

Intelligence artificielle et accélération GPU

1. Principe de sûreté

L'IA ne construit pas directement l'état autoritatif. Elle produit des observations, des candidats, une confiance et des preuves. Le runtime déterministe décide si un candidat est valide. Une ambiguïté matérielle entraîne abstention ou revue humaine.

```
Entree -> extraction -> candidats -> compilation -> execution -> validation
                        |                               |
                        +----- preuves et confiance -----+
```

Une cote explicitement lisible doit être exacte ou rejetée. Une dimension absente ne peut jamais être remplie comme vérité sans règle ou décision.

2. Architectures comparées

Architecture	Forces	Faiblesses	Rôle MORPHOIA
CNN	Localité, coût maîtrisé, OCR/segmentation	Contexte global limité	Détection de traits, symboles, vues, défauts
Vision Transformer	Contexte global et multimodalité	Besoin de données, coût quadratique ou approximations	Plans, photos, vidéos, association cote-entité
Transformer autoregressif	Génération séquentielle, texte/code	Erreurs cumulatives, hallucinations, ordre arbitraire	Proposer une source .morph sous grammaire contrainte
Diffusion	Diversité de candidats et formes	Validation exacte difficile, latence	Hypothèses de géométrie/structure, jamais autorité
GNN	Topologie, adjacence, contraintes, B-rep	Construction du graphe et scaling	Encodage B-rep, DAG, matching d'entités
Modèle multimodal	Fusion image, texte, points et CAO	Alignement, données rares, calibration	Architecture cible de reconstruction
Agent	Appels d'outils, itération avec validateur	Boucles coûteuses, erreurs d'outil	Orchestration bornée et auditable
JEPA/world model	Représentation prédictive sans pixel-level exhaustif	Applications CAO encore exploratoires	Préentraînement de vues/états et cohérence multi-vues

La meilleure architecture n'est probablement pas un modèle unique. Un système composite sépare perception, génération de graphe, exécution et classement.

3. Pipeline multimodal

3.1 Dessins industriels et vues orthographiques

1. rectification, débruitage et segmentation des vues ;
2. OCR du cartouche, des cotes et notes ;
3. détection des traits, axes, hachures, coupes et symboles ;
4. association cote-entité avec graphe biparti ;
5. cohérence entre vues et génération de primitives 3D candidates ;
6. recherche de séquences P1 compatibles ;
7. génération contrainte de plusieurs graphes ;
8. exécution et comparaison aux vues ;
9. abstention si l'inversion reste multiple.

Les vues en coupe et auxiliaires ajoutent des frames explicites. Les annotations ISO et PMI deviennent preuves structurées, pas texte décoratif.

3.2 Croquis manuscrits

La pipeline estime traits, jonctions, symboles et incertitude. Les proportions ne sont pas interprétées comme cotes. Les valeurs textuelles ou décisions utilisateur pilotent la géométrie.

3.3 Photographies et vidéo

Calibration, pose, silhouettes, profondeur multi-vues, NeRF/3DGS ou reconstruction dense produisent un champ de preuves. L'échelle doit venir d'une référence connue. La vidéo aide l'occlusion et la multi-vue, mais ne révèle pas les cavités invisibles ni l'intention.

3.4 Scans et nuages de points

Nettoyage, normales, segmentation, fitting de primitives, symétries et courbes d'intersection produisent surfaces candidates. L'incertitude capteur est conservée séparément des tolérances de conception.

3.5 STL, OBJ, PLY et meshes

Le mesh fournit forme et parfois groupes/matériaux. Une GNN ou un encodeur mesh propose faces fonctionnelles, primitives et ordre de features. La B-rep et l'historique restent des hypothèses validées par écart géométrique et scénarios.

3.6 STEP, IGES, Parasolid et ACIS

La géométrie exacte réduit l'incertitude de forme. Des encodeurs B-rep, signatures et recherche de motifs proposent l'intention. Les formats natifs peuvent révéler davantage via API autorisée, mais leur sémantique reste versionnée.

3.7 Texte et voix

La voix est transcrite avec horodatage et confiance. Un LLM transforme la commande en patch de graphe, affiche le diff, compile et demande confirmation pour toute valeur ou référence ambiguë. Le modèle ne reçoit pas de pouvoir d'I/O arbitraire.

4. Génération contrainte

Trois stratégies sont comparées :

1. Token grammar constrained decoding : le décodeur ne peut produire qu'un token syntaxiquement valide. Simple et immédiatement testable.
2. AST/graph generation : le modèle génère des noeuds typés et des arêtes. Meilleur contrôle sémantique, entraînement plus complexe.
3. Tool-using agent : le modèle ajoute ou corrige un noeud après diagnostics. Utile pour raffinement, mais borné en nombre d'étapes.

Le choix initial combine 1 et 3. Le graphe généré passe ensuite par types, unités, DAG, backend et scénarios.

5. Objectifs d'apprentissage

Le score ne se limite pas à Chamfer ou IoU. Une fonction multi-objectif peut inclure :

- validité syntaxique ;
- validité du graphe ;
- satisfaction des cotes et contraintes ;
- validité B-rep ;
- propriétés de masse ;
- distance géométrique ;
- stabilité des références ;
- scénarios d'édition ;
- conservation PMI ;
- nombre de pertes ;
- coût de correction humaine ;
- calibration et abstention.

Les contraintes dures ne deviennent pas une simple pénalité molle : un candidat non conforme est rejeté.

6. Données nécessaires

6.1 Unité de donnée

Une unité idéale contient :

- sources originales ;
- graphe de construction et versions ;
- B-rep/snapshots ;
- paramètres et contraintes ;
- PMI et matériaux ;
- assemblages/configurations si profil ;
- scénarios de modification ;
- liens de provenance ;
- droits d'entraînement et de publication ;
- corrections humaines et temps.

6.2 Corpus

Niveau	Fondation	MVP	1.0
Microcas noyau	1 000	10 000	50 000
Pièces P1/P2	500-2 000	10 000+	50 000+
Scénarios d'édition	5+ par pièce	50 000+	250 000+
Cas adversariaux	250	2 000	10 000
Backends/oracles	2	2 noyaux + 2 CAO	2 noyaux + 4 CAO

Les splits se font par famille de pièce, fournisseur et date. Des variantes proches ne doivent pas traverser train/test. Le jeu industriel final est aveugle.

6.3 Sources existantes

Fusion 360 Gallery, DeepCAD, SketchGraphs, ABC, Fusion 360 Reconstruction, CADFS et BenchCAD sont utiles pour préentraînement et comparaison. Aucun ne remplace un corpus légal réunissant historique, PMI, assemblages et scénarios industriels.

7. Entraînement

Etape A - préentraînement

Préentraînement auto-supervisé sur vues, B-rep, points et texte. Les objectifs possibles sont masked modeling, contrastive alignment, prédiction de relations et JEPA multi-vues.

Etape B - supervision structurée

Apprentissage de séquences et graphes P1 avec curriculum : sketch simple, extrusion, perçage, motifs, puis références et PMI.

Etape C - exécution dans la boucle

Les candidats sont compilés et exécutés. Les diagnostics deviennent supervision. Cette étape doit sandboxer chaque programme et limiter temps/mémoire.

Etape D - préférences ingénieur

Classement par lisibilité, intention, stabilité après édition et temps de correction. Les préférences ne peuvent corriger une violation dimensionnelle ou PMI.

Etape E - calibration

Temperature scaling, conformal prediction ou méthodes équivalentes calibrent acceptation, revue et abstention. Cible expérimentale ECE au plus 0,05 sur domaine validé.

8. Métriques IA

Métrique	Cible expérimentale MVP
Source syntaxiquement valide sous décodage contraint	98 % ou plus
Dimensions explicites automatiquement acceptées	100 % exactes ou abstention
Détection hors profil	95 % ou plus
Rappel top-5 de graphes candidats	90 % ou plus sur P1 ciblé
ECE de confiance	0,05 ou moins
Ambiguïtés silencieuses	0
Scénarios d'édition réussis sur acceptés	95 % ou plus
Temps humain médian économisé	40 % ou plus

Ces valeurs sont des gates à tester, pas des performances acquises.

9. Carte CPU/GPU

Tâche	CPU	GPU	Autorité initiale
OCR/vision	Possible	Très favorable	GPU avec validation
Encodeur point/mesh/B-rep échantillonnée	Lent à grande échelle	Très favorable	GPU
LLM/multimodal	Peu compétitif	Tensor Cores	GPU
Nearest-neighbor/Chamfer	Possible	Très favorable	GPU non autoritatif
Rendu/ray tracing	Possible	RTX/OptiX favorable	Dérivé
Tessellation en lots	Possible	Favorable selon pipeline	Dérivé ou contrôlé
Solveur d'esquisse	Robuste/mature	Gain incertain, divergence	CPU qualifié
Booléens/fillets B-rep	Noyaux matures	Recherche, divergence forte	CPU exact
Persistent naming	Branching et graphes	Accélération partielle	CPU qualifié
PMI et règles	Faible coût	Inutile initialement	CPU

10. Technologies GPU

CUDA et Tensor Cores

Voie de performance initiale pour entraînement, inférence, points, distances et batching. TensorRT ou ONNX Runtime peuvent servir l'inférence. Les kernels custom utilisent CUDA uniquement après profilage.

RTX et OptiX

Ray casting, visibilité, génération de vues, occlusions et rendu synthétique. Les résultats sont des preuves ou vues dérivées, pas une topologie exacte.

Vulkan Compute

Option portable pour visualisation et kernels simples côté client. Son adoption dépend d'un gain par rapport au rendu existant et du coût de maintenance shader.

DirectML

Déploiement Windows multi-vendeur pertinent pour certaines CAO clientes. Priorité après preuve d'un besoin desktop.

OpenCL

Large portabilité historique, mais écosystème ML moderne moins cohérent. Maintenu comme option d'intégration, non priorité MVP.

ROCm

Option serveur AMD pour entraînement/inférence. Ne doit pas être financée en parallèle de CUDA avant au moins deux besoins clients ou infrastructurels.

SYCL/oneAPI

Option Intel/HPC et portabilité C++. Appropriée pour kernels ciblés si le benchmark et les pilotes la justifient.

11. Estimations de performance

Les estimations suivantes sont hypothèses à mesurer sur corpus identique :

Sous-système	Hypothèse de gain GPU	Risque
Inférence vision/transformer en batch	x5 a x50 vs CPU serveur	Dépend du modèle et batch
Distance points/mesh	x10 a x100	Transferts mémoire et précision
Ray casting multi-vues RTX	x10 a x100	Scène et cohérence drivers
Tessellation massive	x2 a x20	Noyau peut rester dominant
Booléens B-rep exacts	Pas de gain assumé	Divergence et cas dégénérés

Un portage n'est accepté que si le protocole mesure p50, p95, mémoire, précision, déterminisme et taux d'échec. Une cible de gain p95 x2 sans perte est raisonnable pour décider un portage local, mais ne vaut pas seuil universel.

12. Portabilité ML

Les modèles sont entraînés avec PyTorch ou équivalent, exportés vers ONNX lorsque la couverture opérateur le permet, puis déployés via ONNX Runtime, TensorRT, OpenXLA/IREE ou TVM selon plateforme. Les opérateurs géométriques spécifiques nécessitent une Component Justification Record.

13. Sécurité MLOps

- provenance et licence de chaque dataset ;
- modèle, tokenizer et runtime épinglés ;
- poids et artefacts signés ;
- aucune donnée client réutilisée sans droit ;
- isolation des workers ;
- prompts et sorties journalisés selon politique de confidentialité ;
- évaluation hors distribution ;
- red teaming des fichiers et programmes malveillants ;
- possibilité de déploiement souverain ou hors ligne.

Guide utilisateur de MORPHOIA

1. Statut à lire avant toute utilisation

MORPHOIA est actuellement un candidat expérimental 0.1. La façade textuelle `.morph`, son modèle de données et ses profils peuvent changer ou être abandonnés. Ils ne constituent ni une norme, ni un format CAO stable, ni un outil certifié.

Deux versions apparaissent actuellement dans le dépôt :

Élément	Version actuelle	Signification
Façade textuelle et IR canonique	0.1	Version expérimentale acceptée dans les fichiers <code>.morph</code>
Paquet Python et CLI	0.2.0-dev	Version de développement de l'implémentation de référence

Un document doit donc commencer par :

```
morphoia 0.1;
```

La commande `morphoia --version` affiche la version du paquet, pas celle de la façade. Cette distinction est volontaire pendant le prototypage.

La Phase 1 impose de réutiliser STEP/AP242, les ressources ISO 10303, OCCT et les solveurs existants avant de créer un composant nouveau. Le prototype actuel sert à évaluer une syntaxe et un graphe de construction ; il ne démontre pas encore la portabilité géométrique industrielle.

2. Ce que le prototype sait réellement faire

Implémenté

- analyser lexicalement et syntaxiquement un document MORPHOIA 0.1 ;
- produire des diagnostics localisés, en texte ou en JSON ;
- vérifier une partie des symboles, profils, unités et dépendances ;
- refuser les cycles détectés dans le graphe de déclarations ;
- contrôler le contrat minimal des références topologiques persistantes ;
- distinguer les quatre catégories de tolérances ;
- imposer une preuve aux déclarations marquées `hypothesis`, `ambiguous` ou

`inferred` ;

- compiler le document vers un graphe canonique JSON déterministe ;
- calculer un hash sémantique SHA-256 ;
- représenter séparément les résultats de résolution topologique unique,

`ambiguous` et `missing` dans l'API Python ;

- gérer en mémoire des révisions immuables avec transaction, commit atomique, rollback, undo/redo et rejet des écritures concurrentes obsolètes ;
- représenter des preuves, décisions et abstentions avec contrôles minimaux ;

- produire un registre machine-readable des pertes et bloquer un commit applicatif

lorsque ce registre contient une perte `error` ou `fatal`.

Pas encore implémenté

- exécution des opérations par Open CASCADE ou un autre noyau géométrique ;
- création ou validation d'une B-rep ;
- solveur d'esquisse et de contraintes ;
- import ou export STEP/AP242, IGES, Parasolid, ACIS, FreeCAD ou glTF ;

- validation géométrique, topologique, PMI/GD&T ou industrielle ;
- exécution réelle des scénarios d'édition ;
- persistance durable des révisions transactionnelles ;
- raccordement automatique du registre des pertes aux commandes CLI ;
- visualisation 3D ;
- compilation inverse d'un modèle CAO vers une histoire paramétrique ;
- backend GPU, inférence IA ou API asynchrone ;
- résolution et téléchargement des imports ou extensions.

Une source peut donc être déclarée « valide » par le prototype tout en décrivant une opération géométriquement impossible. Le verdict actuel porte sur la syntaxe et le sous-ensemble sémantique implémenté, pas sur la fabricabilité ni sur une B-rep.

3. Installation

3.1 Prérequis

- Python 3.12 ou plus récent ;
- Git ;
- le dépôt MORPHOIA.

```
git clone https://github.com/aminoside/morphoia.git
cd morphoia
python -m venv .venv
source .venv/bin/activate
python -m pip install -e .
```

Sous PowerShell, l'activation de l'environnement virtuel est :

```
.venv\Scripts\Activate.ps1
```

Pour contribuer et exécuter les outils de développement :

```
python -m pip install -e '[dev]'
```

Il est aussi possible d'utiliser directement un checkout sans installation :

```
PYTHONPATH=src python -m morphoia --version
```

4. Premier parcours

4.1 Valider l'exemple fourni

Depuis la racine du dépôt :

```
morphoia validate examples/mounting_plate.morph
```

Résultat attendu :

```
examples/mounting_plate.morph: valid
```

Sans installation éditable :

```
PYTHONPATH=src python -m morphoia validate examples/mounting_plate.morph
```

La commande retourne le code de sortie 0 si le sous-ensemble contrôlé est valide et 1 si au moins une erreur est détectée.

4.2 Compiler vers le graphe canonique JSON

```
morphoia compile examples/mounting_plate.morph \
-o build/mounting_plate.mcir.json
```

La commande crée le dossier de sortie si nécessaire, valide d'abord la source puis écrit un document JSON trié. Elle affiche le hash sémantique :

```
wrote build/mounting_plate.mcir.json (<sha256>)
```

Ce JSON est une IR de débogage et d'échange entre composants du prototype. Ce n'est ni un fichier STEP, ni une B-rep, ni un modèle visualisable.

4.3 Obtenir des diagnostics lisibles par une machine

```
morphoia validate examples/mounting_plate.morph --json
```

La sortie contient :

- le chemin du fichier ;
- le booléen valid ;
- le code, la sévérité et le message de chaque diagnostic ;
- la ligne et la colonne lorsqu'elles sont disponibles ;
- une suggestion facultative dans hint.

5. Écrire un document 0.1

5.1 En-tête, namespace et modèle

```
morphoia 0.1;
namespace org.example.parts;

model Spacer [
  profile = "P1",
  id = "20000000-0000-4000-8000-000000000001"
] {
  // déclarations
}
```

Le namespace est facultatif. Un document doit contenir au moins un modèle. Le profil par défaut est P1 ; il est néanmoins préférable de le déclarer explicitement.

5.2 Paramètres et unités

```
parameter diameter: length = 20 mm [
  min = 5 mm,
  max = 100 mm,
  status = observed,
  evidence = "drawing:D1"
];
parameter angle_a: angle = 45 deg;
```

Le registre minimal connaît notamment nm, um, mm, cm, m, in, ft, rad, deg, mg, g, kg, ms, s, min, N, Pa et MPa.

Le typage dimensionnel est encore partiel. Le validateur contrôle en particulier la dimension d'un littéral affecté à un paramètre :

```
parameter width: length = 10 deg; // MORPH-E110
```

Les unités composées, conversions avancées et types géométriques complets relèvent encore de la cible.

5.3 Les quatre tolérances non interchangeables

```
tolerance build_precision: kernel_tolerance = 0.001 mm;
tolerance scan_precision: measurement_uncertainty = 0.05 mm;
tolerance drawing_size: dimensional_tolerance = 0.10 mm;
tolerance position_zone: gdt_zone = 0.20 mm;
```

Le prototype refuse tout autre nom de catégorie avec MORPH-E140. Cette séparation évite d'utiliser une tolérance fonctionnelle comme marge numérique de réparation.

5.4 Esquisse et feature

La syntaxe réellement acceptée par le parser actuel utilise un bloc pour chaque feature :

```
datum base_plane: plane = world.xy;

sketch base on base_plane {
  profile disk = circle(diameter / 2);
}

feature body: extrude {
  profile = base.disk;
  distance = thickness;
  direction = +world.z;
  result = solid;
}
```

Le parser conserve les affectations du bloc. Il ne vérifie pas encore toutes les préconditions propres à extrude et n'exécute pas l'opération.

5.5 Référence topologique persistante

```
reference top_face: face = select(
  body.faces,
  role = "cap.end",
  normal = world.z,
  cardinality = unique
);
```

Le validateur exige actuellement :

- un appel `select(...)` ;
- une collection source positionnelle ;
- un rôle sémantique ;
- `cardinality = unique` OU `one` ;
- au moins une preuve parmi `normal`, `surface`, `area`, `radius`, `adjacent` ou

signature.

Le prototype vérifie le contrat de la requête, mais ne dispose pas encore des faces d'une B-rep pour l'évaluer. Le module Python `morphoia.topology` contient séparément le résolveur à trois états utilisé par les tests.

5.6 Provenance et incertitude

```
parameter inferred_width: length = 80 mm [
  status = inferred,
  evidence = "drawing:D1:dimension-7"
];
```

Sans attribut `evidence`, les états `hypothesis`, `ambiguous` et `inferred` produisent MORPH-E150. Ce mécanisme ne transforme pas une estimation en vérité : la décision et la validation déterministe restent des fonctions cibles.

5.7 Scénarios d'édition

```
validate {
  assert manifold(body);
  scenario Wider {
    set diameter = 30 mm;
    expect valid(body);
  }
}
```

Le parser et le compilateur conservent ces nœuds. Le runtime actuel ne régénère pas encore la géométrie et n'évalue donc pas réellement les assertions.

6. Profils actuels

Profil	Opérations reconnues par le validateur	Réalité d'exécution
P1	extrude, cut, revolve, boolean, hole, pattern, mirror, fillet, chamfer	Non exécutées
P2	P1 + thread	Non exécutées
P3	Hérite de P2 ; blocs assembly parsables	Aucun moteur d'assemblage
P4	P3 + sweep, loft, blend, shell, draft	Non exécutées

Le fait qu'une opération soit reconnue signifie seulement qu'elle n'est pas rejetée comme hors profil. Cela ne constitue pas une implémentation géométrique.

7. Comprendre les diagnostics

Code	Contrôle actuellement associé
MORPH-E000	caractère, commentaire ou chaîne invalide
MORPH-E010	syntaxe invalide
MORPH-E001	version de façade autre que 0.1
MORPH-E100	profil inconnu
MORPH-E101	déclaration de modèle dupliquée
MORPH-E102	référence inconnue
MORPH-E103	cycle de dépendances
MORPH-E110	dimension incompatible sur un paramètre contrôlé
MORPH-E120	opération non autorisée par le profil
MORPH-E130..134	contrat de référence persistante incomplet
MORPH-E140	catégorie de tolérance invalide
MORPH-E150	hypothèse ou inférence sans preuve

Ces contrôles sont volontairement incomplets. Une absence de diagnostic n'est pas une certification.

8. Utiliser l'API Python

```
from pathlib import Path

from morphoia import RuntimeStore, canonical_json, compile_document, validate_source

source = Path("examples/mounting_plate.morph").read_text(encoding="utf-8")
result = validate_source(source)

if not result.ok or result.document is None:
    for diagnostic in result.diagnostics:
        print(diagnostic.code, diagnostic.message)
    raise SystemExit(1)

ir = compile_document(result.document)
print(ir["semantic_sha256"])
print(canonical_json(ir))

store = RuntimeStore(ir)
transaction = store.begin()
transaction.graph["review_status"] = "reviewed"
revision = transaction.commit("record review status")
print(revision.number, revision.semantic_sha256)
```

Utiliser `validate_source` ou `validate_file` pour les entrées non fiables. La fonction parse lève directement `LexError` ou `ParseError`.

`RuntimeStore` est uniquement un magasin en mémoire servant de contrat exécutable pour l'atomicité. Il ne lance pas de backend et ne persiste pas ses révisions entre deux processus.

9. Bonnes pratiques pendant la phase expérimentale

- conserver `morphoia 0.1`; tant que le parser ne supporte aucune autre version ;
- attribuer un UUID explicite aux modèles et déclarations qui doivent survivre à un renommage ;
- déclarer unités, profil, tolérances et provenance ;
- ne jamais employer un numéro de face de noyau comme identité persistante ;
- exiger une cardinalité unique au lieu de choisir silencieusement un candidat ;
- versionner la source et le JSON canonique utile aux tests, mais ne pas présenter ce dernier comme une B-rep ;
- conserver séparément tout STEP ou résultat géométrique de référence ;
- considérer chaque import ou extension comme non résolu dans le prototype actuel ;

- accompagner tout composant nouveau d'un Component Justification Record.

10. Gates hérités de la Phase 1

Gate	Preuve requise	Statut
G0	Spécification, implémentation de référence, 100 cas golden	En cours
G1	P1/P2 sur deux backends, B-rep valide ≥ 99 %, scénarios ≥ 95 %	Non atteint
G2	Corpus et conformité publics, registre des pertes, seconde implémentation	Non atteint
G3	Gain médian de temps humain ≥ 40 % sur deux pilotes	Non atteint
G4	G1-G3, gouvernance externe et politique brevets	Interdit avant preuves

Tant que ces gates ne sont pas franchis, les termes standard, stable, certified et 1.0 ne doivent pas qualifier MORPHOIA.

11. Documents associés

- [Spécification candidate](language-specification.md)
- [Grammaire actuellement implémentée](grammar.ebnf)
- [Catalogue des opérations](operation-catalog.md)
- [Guide développeur](developer-guide.md)
- [Tutoriels et FAQ](tutorials-faq.md)
- [Exigences et gates](requirements.md)

Guide développeur de MORPHOIA

1. Portée et statut

Ce guide décrit l'implémentation présente dans `src/morphoia`, et non le runtime industriel visé par les documents d'architecture.

La façade source et l'IR sont un candidat expérimental 0.1. Le paquet Python est actuellement en version 0.2.0-dev. Aucune API n'est stable et aucun succès des tests unitaires ne vaut conformité CAO, STEP ou industrielle.

La règle de Phase 1 est opposable à toute contribution : un composant nouveau n'est acceptable qu'après démonstration d'un manque ou d'un avantage mesurable. Il ne faut notamment réécrire ni noyau B-rep, ni solveur de contraintes, ni ontologie STEP.

2. Préparer l'environnement

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
python -m unittest discover -s tests -v
```

Le projet exige Python 3.12 ou plus récent. Les modules du cœur n'ont actuellement pas de dépendance d'exécution externe.

Sans installation :

```
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python -m morphoia validate examples/mounting_plate.morph
```

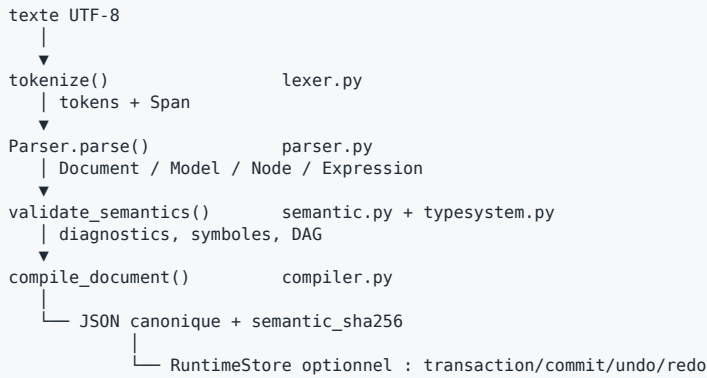
3. Arborescence du prototype

```
src/morphoia/
├── __init__.py           # API publique et version du paquet
├── __main__.py          # python -m morphoia
├── cli.py               # commandes valider et compiler
├── lexer.py             # lexer sans dépendance
├── parser.py            # parser descendant récursif 0.1
├── model.py             # AST, spans et diagnostics
├── typesystem.py        # inférence dimensionnelle minimale
├── semantic.py          # symboles, profils, DAG et règles sémantiques
├── validator.py         # façade de validation sûre
├── compiler.py          # lowering JSON et hash canonique
├── topology.py          # résolution unique/ambiguë/manquante
├── provenance.py        # preuves, décisions et abstention
├── losses.py            # registre machine-readable des pertes
├── runtime.py           # transactions et révisions immuables en mémoire
├── backends/
│   ├── base.py          # contrat abstrait de backend
│   └── json_backend.py  # sérialisation JSON canonique
└── examples/
    └── mounting_plate.morph

schemas/
└── morphoia-ir-0.1.schema.json

tests/
├── test_parser.py
├── test_runtime.py
└── test_topology.py
```

4. Pipeline réellement exécuté



La CLI compile écrit directement le dictionnaire produit par `compile_document`. La classe `CanonicalJsonBackend` existe comme démonstrateur du contrat de backend, mais la CLI ne possède pas encore de registre ni de sélection de backends.

Aucun nœud n'est transmis à un noyau géométrique et aucun scénario n'est exécuté.

5. Modules et responsabilités

5.1 `model.py`

Le modèle syntaxique repose sur des dataclasses typées :

- `Span` utilise des lignes et colonnes indexées à partir de 1 ;
- `Diagnostic` porte code, sévérité, message, span et hint ;
- les expressions sont `Name`, `Literal`, `Call`, `Unary`, `Binary` et

`ListExpression` ;

- `Node` représente uniformément déclaration, bloc ou instruction ;
- `Document` contient version, namespace, imports et modèles ;
- `ValidationResult.ok` est faux dès qu'un diagnostic de sévérité `ERROR` existe.

Les nœuds syntaxiques sont immuables (`frozen=True`, `slots=True`). Cette propriété réduit les mutations accidentelles avant canonicalisation.

5.2 `lexer.py`

Le lexer reconnaît :

- identifiants ASCII ;
- nombres entiers ou décimaux avec exposant ;
- chaînes à échappements JSON ;
- commentaires `//` et `/ ... /` non imbriqués ;
- opérateurs simples et doubles ;
- positions précises dans le texte.

Il ne résout pas le registre d'unités. Le parser traite actuellement un nombre suivi d'un identifiant comme une quantité ; le type system détermine ensuite si l'unité est connue.

5.3 `parser.py`

Le parser implémente la grammaire publiée dans `docs/phase2/grammar.ebnf` :

- en-tête `morphoia 0.1` ;
- namespace et imports ;
- un ou plusieurs modèles ;

- paramètres, constantes, datums, références, matériaux, PMI et tolérances ;
- sketches, features, validation, scénarios et assemblages ;
- attributs [key = value] ;
- expressions avec précédence ;
- nœuds génériques destinés aux expérimentations d'extensions.

La syntaxe réelle des arguments nommés est `name = value`, pas `name: value`. La syntaxe réelle des features est :

```
feature body: extrude {
  profile = base.outer;
  distance = thickness;
}
```

Le fallback générique préserve un verbe inconnu dans l'AST. Cela ne signifie pas qu'une extension est résolue ou exécutable.

5.4 typesystem.py

Le type system est volontairement minimal. Il connaît un registre d'unités, quelques alias et certains types de retour d'appels. Il vérifie aujourd'hui surtout le type d'une expression affectée à un paramètre lorsque l'inférence est certaine.

Il ne faut pas documenter comme implémentés :

- l'algèbre dimensionnelle complète ;
- les unités composées ;
- les types génériques de références ;
- la covariance géométrique ;
- les signatures complètes des opérations.

5.5 semantic.py

Les règles actuelles sont :

- seule la version source 0.1 est acceptée ;
- profils P1 à P4 reconnus ;
- symboles de premier niveau uniques ;
- références de symboles connues ;
- cycles de dépendances refusés ;
- opération autorisée par le profil ;
- référence persistante fondée sur `select` avec source, rôle, preuve et cardinalité

unique ;

- quatre catégories de tolérances uniquement ;
- preuve obligatoire pour hypothèse, ambiguïté ou inférence.

Le DAG est déduit des références de noms, puis ordonné de manière déterministe en triant les dépendances. Les corps imbriqués sont parcourus pour trouver les expressions, mais le symbol table principal reste celui du modèle.

5.6 compiler.py

Le compilateur transforme l'AST validé en dictionnaire JSON neutre :

- format `morphoia.canonical-construction-graph` ;
- version d'IR 0.1 ;
- source, namespace, imports, modèles et déclarations ;
- ordre de dépendances ;
- contrat d'exécution et ancrages ISO ;

- hash sémantique.

Les clés d'arguments et d'attributs sont triées. Les décimaux sont sérialisés en chaînes accompagnées de `numeric_encoding = "decimal"`.

Un attribut `[id = "..."]` devient l'ID canonique. Sans ID explicite, un UUIDv5 est dérivé du namespace et du chemin. Un renommage change donc l'ID dérivé : les actifs destinés au round-trip doivent employer un UUID explicite.

`semantic_hash()` retire un éventuel champ `semantic_sha256`, sérialise le reste en JSON compact avec clés triées, puis calcule SHA-256. La stabilité n'est garantie que pour le même contrat et la même version du prototype ; elle n'est pas encore une garantie inter-implémentations.

5.7 topology.py

Ce module est un contrat exécutable indépendant d'une B-rep :

```
from morphoia.topology import (
    EntityCandidate,
    ReferenceQuery,
    ResolutionStatus,
    resolve_reference,
)

query = ReferenceQuery(
    kind="face",
    producer="blank",
    role="cap.end",
    signature=((("normal", "+Z")),),
)

candidate = EntityCandidate(
    identity="face-a",
    kind="face",
    producer="blank",
    role="cap.end",
    signature=((("normal", "+Z")),),
)

result = resolve_reference(query, [candidate])
assert result.status is ResolutionStatus.UNIQUE
assert result.entity.identity == "face-a"
```

Le filtre utilise type, producteur, rôle, signature et adjacence. Les candidats sont triés seulement pour rendre le diagnostic déterministe. Plusieurs résultats restent AMBIGUOUS ; le tri n'autorise jamais un choix silencieux.

5.8 provenance.py

Le contrat minimal de confiance distingue :

- les preuves `drawing`, `pmi`, `cad_model`, `image`, `point_cloud`, `mesh`, `text`,

`human_decision` et `derived_rule` ;

- les décisions `candidate`, `accepted`, `rejected` et `abstained` ;
- l'identité, l'URI, le digest SHA-256 et le locator d'une preuve ;
- confiance, acteur/règle et justification d'une décision.

Une preuve exige un SHA-256 minuscule de 64 caractères. Une décision acceptée exige au moins une preuve et un `decided_by`. La confiance, si elle existe, appartient à l'intervalle fermé `[0, 1]`.

Ce module est un contrat minimal issu de la Phase 1, pas une ontologie universelle de provenance.

5.9 losses.py

`LossRecord` classe une perte de géométrie, topologie, historique paramétrique, contrainte, PMI, matériau, assemblage, provenance, identité ou comportement. Les sévérités sont `info`, `warning`, `error` et `fatal`.

`LossRegister.blocks_commit` devient vrai si au moins une perte est `error` ou `fatal`. `to_dict()` produit le schéma `morphoia.loss-register/0.1`. Le registre n'est pas encore branché automatiquement aux commandes ou backends.

5.10 runtime.py

`RuntimeStore` fournit un historique en mémoire de graphes immuables :

- `begin()` ouvre une transaction sur la révision courante ;

- `commit()` exige un message et exécute les validateurs fournis avant mutation ;
- `rollback()` ferme sans commit ;
- `undo()` et `redo()` déplacent le curseur ;
- une transaction basée sur une ancienne révision lève `RevisionConflict` ;
- un commit après undo tronque la branche redo ;
- chaque révision reçoit un nouveau hash sémantique.

Le store copie profondément les graphes pour éviter la mutation d'une révision commise. Il ne fournit ni persistance, ni journal distribué, ni fusion de branches. La géométrie demeure la responsabilité des backends.

5.11 backends/

`BackendCapabilities` décrit nom, version, profils, opérations, B-rep exacte, PMI, mode déterministe et licence. `BackendResult` sépare succès, URI d'artefact, propriétés de validation, pertes et diagnostics.

Le seul backend concret est `CanonicalJsonBackend`. Il sérialise l'IR sans perte, mais ne produit aucune géométrie. `exact_brep` vaut donc `False`.

6. API Python publique actuelle

Les symboles exportés par `morphoia` sont :

```
from morphoia import (
    canonical_json,
    compile_document,
    parse,
    semantic_hash,
    validate_file,
    validate_source,
    RuntimeStore,
)
```

Usage sûr :

```
from morphoia import compile_document, validate_file

result = validate_file("examples/mounting_plate.morph")
if not result.ok or result.document is None:
    raise ValueError([item.message for item in result.diagnostics])

ir = compile_document(result.document)
```

`compile_document` suppose un document déjà validé. Il ne relance pas lui-même le validateur.

7. CLI actuelle

Deux sous-commandes seulement sont disponibles :

```
morphoia validate SOURCE [--json]
morphoia compile SOURCE -o OUTPUT
```

`validate` retourne 0 ou 1. `compile` affiche d'abord les diagnostics en texte, refuse une source invalide puis écrit le JSON. Les commandes `import`, `export`, `render`, `step`, `occt`, `benchmark` et `serve` ne sont pas encore implémentées.

Pour ajouter une commande :

1. écrire une fonction `command_<name>(arguments) -> int` dans `cli.py` ;
2. enregistrer son sous-parser dans `build_parser()` ;
3. conserver les codes de sortie 0 succès, 1 erreur métier ;
4. fournir une sortie machine-readable si la commande produit des diagnostics ;
5. ajouter des tests de CLI avant de la documenter comme disponible.

8. Ajouter ou modifier une construction syntaxique

Ordre recommandé :

1. vérifier que la construction est autorisée par un CJR ou correspond à un concept

STEP déjà retenu ;

2. mettre à jour la grammaire candidate ;
3. ajouter les tokens strictement nécessaires dans `lexer.py` ;
4. ajouter ou spécialiser le parsing dans `parser.py` ;
5. enrichir les dataclasses seulement si `Node` ne peut préserver la sémantique ;
6. ajouter les règles de symboles, types et profils ;
7. définir le lowering canonique ;
8. ajouter tests valides, invalides et de hash ;
9. documenter séparément « parsé », « validé » et « exécuté ».

Le parser générique ne doit pas servir à prétendre qu'une opération est supportée.

9. Ajouter une unité ou une fonction typée

Pour une unité simple :

1. ajouter son symbole et sa dimension dans `UNIT_DIMENSIONS` ;
2. ajouter des tests positifs et d'incompatibilité ;
3. documenter facteur de conversion et standard de référence avant toute

canonicalisation SI réelle.

Pour une fonction connue du type `system`, ajouter son type dans `CALL_RETURN_TYPES`. Cette table ne vérifie encore ni les paramètres ni les unités dérivées ; toute extension substantielle doit faire évoluer le modèle de signatures plutôt que multiplier des chaînes spéciales.

10. Ajouter une opération à un profil

Les opérations reconnues se trouvent dans `PROFILE_OPERATIONS`. Une opération P2 hérite automatiquement des opérations P1 ; P4 hérite de tous les profils précédents.

Avant ajout, fournir :

- profil minimal ;
- préconditions et postconditions ;
- arguments affectant la géométrie ;
- rôles topologiques ;
- cas dégénérés ;
- mapping STEP ou écart démontré ;
- fallback et registre des pertes ;
- scénarios d'édition ;
- CJR si le composant ou la sémantique est nouveau.

L'ajout dans `PROFILE_OPERATIONS` ne constitue qu'une autorisation syntaxique. Une implémentation backend et des tests géométriques seront nécessaires pour parler de support réel.

11. Ajouter un backend

Un backend expérimental implémente `Backend` :

```
from morphoia.backends.base import Backend, BackendCapabilities, BackendResult

class ExampleBackend(Backend):
    def capabilities(self) -> BackendCapabilities:
        return BackendCapabilities(
            name="example",
            version="0.1",
            profiles=("P1",),
            operations=("extrude",),
            exact_brep=False,
            pmi=False,
```

```

        deterministic_mode=True,
        license="declare-me",
    )

    def execute(self, canonical_ir: dict[str, object]) -> BackendResult:
        return BackendResult(
            success=False,
            artifact_uri=None,
            validation_properties={},
            losses=(),
            diagnostics=("not implemented",),
        )

```

Un backend OCCT futur doit rester derrière cette frontière : aucun type OCCT ne doit fuir dans l'IR canonique. Il devra publier capacités, environnement verrouillé, propriétés, événements de healing et pertes. La Phase 1 interdit de remplacer OCCT par un nouveau noyau généraliste.

12. Tests

La suite actuelle contient quatorze tests unitaires couvrant :

- validité de l'exemple ;
- déterminisme du hash dans la même implémentation ;
- référence inconnue ;
- incompatibilité dimensionnelle ;
- cardinalité obligatoire ;
- résolution topologique unique, ambiguë et manquante ;
- commit, rollback logique, undo/redo et fermeture des transactions ;
- rejet d'une transaction obsolète ;
- absence de mutation après échec d'un validateur de commit ;
- preuve et acteur obligatoires pour une décision acceptée ;
- sérialisation et blocage d'un registre de pertes.

Exécution :

```
python -m unittest discover -s tests -v
```

Après installation des dépendances de développement :

```
python -m ruff check src tests
python -m pytest
```

Toute correction doit ajouter un test de non-régression. Toute évolution du hash ou de l'IR doit être considérée comme une modification de contrat et explicitée.

13. Diagnostics

Les codes doivent rester stables et structurés :

- lexing MORPH-E000 ;
- parsing MORPH-E010 ;
- version MORPH-E001 ;
- sémantique MORPH-E100 et suivants.

Un diagnostic doit inclure un span aussi précis que possible et un `hint` seulement s'il propose une correction sûre. Ne jamais transformer une ambiguïté mécanique en warning si elle peut changer le résultat.

14. Frontières à ne pas franchir silencieusement

Fonction	Statut actuel	Condition avant annonce de support
Lexer/parser 0.1	Implémenté	Corpus golden G0
Validation sémantique	Partielle	Règles et tests exhaustifs par profil
JSON canonique	Implémenté comme POC	Deux implémentations et bijectivité
Références topologiques	Contrat et tests unitaires	Lignée réelle multi-backend et scénarios
Transactions	Store immuable en mémoire	Persistance, intégration backend et tests de panne
Provenance et pertes	Primitives et tests unitaires	Intégration source/IR/adaptateurs et schémas de conformité
OCCT	Interface seulement	Adaptateur, B-rep, propriétés et tests
Solveur	Non implémenté	Évaluation FreeCAD/SolveSpace/D-Cubed
STEP/AP242	Ancrage déclaré seulement	Mapping et round-trip mesurés
PMI/assemblage	Syntaxe générique seulement	Modèle, backend et conformité P2/P3
IA/GPU	Non implémenté	Pipeline, benchmark et validation

15. Component Justification Record

Avant tout composant nouveau, copier decisions/CJR-TEMPLATE.md et documenter :

- besoin non couvert ;
- alternatives existantes et licences ;
- benchmark reproductible ;
- avantage ou échec démontré ;
- mapping STEP et pertes ;
- coût et stratégie d'abandon.

Une API plus agréable n'est pas, à elle seule, une justification pour réinventer une brique existante.

16. Gates de développement

- G0, en cours : spécification, implémentation de référence et 100 cas golden.
- G1, non atteint : P1/P2 sur deux backends, B-rep valide ≥ 99 %, scénarios ≥ 95 %.
- G2, non atteint : conformité publique, registre des pertes et seconde

implémentation.

- G3, non atteint : gain humain médian ≥ 40 % sur deux pilotes.
- G4, interdit avant preuves : candidat à la standardisation.

Le numéro 1.0 et les qualificatifs stable, standard ou certified ne doivent pas être employés avant les gates correspondants.

Tutoriels et FAQ de MORPHOIA

Préambule

Ces tutoriels concernent le candidat expérimental MORPHOIA 0.1 et le prototype Python actuel. Ils montrent le parsing, la validation partielle et la compilation vers JSON. Ils ne génèrent pas encore de solide CAO.

La version affichée par la CLI peut être 0.2.0-dev : c'est la version du paquet, alors que morphoia 0.1; désigne la version de la façade source et de l'IR.

Tutoriel 1 - Valider et compiler l'exemple officiel

Étape 1 : installer le checkout

```
python -m venv .env
source .env/bin/activate
python -m pip install -e .
```

Étape 2 : vérifier la version du paquet

```
morphoia --version
```

Étape 3 : valider

```
morphoia validate examples/mounting_plate.morph
```

La sortie attendue est :

```
examples/mounting_plate.morph: valid
```

Étape 4 : compiler vers l'IR

```
morphoia compile examples/mounting_plate.morph \
-o build/mounting_plate.mcir.json
```

Étape 5 : inspecter le JSON

```
python -m json.tool build/mounting_plate.mcir.json
```

Rechercher notamment :

- format = morphoia.canonical-construction-graph ;
- format_version = 0.1 ;
- dependency_order ;
- execution_contract ;
- semantic_sha256.

Ce fichier décrit le graphe syntaxique et sémantique connu du prototype. Il ne contient aucune B-rep évaluée.

Tutoriel 2 - Écrire une pièce minimale

Créer un fichier spacer.morph avec :

```
morphoia 0.1;
namespace org.example;

model Spacer [
  profile = "P1",
  id = "20000000-0000-4000-8000-000000000001"
] {
  units {
    length = mm;
    angle = deg;
  }

  parameter diameter: length = 20 mm;
  parameter thickness: length = 5 mm;

  datum base_plane: plane = world.xy;

  sketch base on base_plane {
    profile disk = circle(diameter / 2);
  }
}
```

```
feature body: extrude {
  profile = base.disk;
  distance = thickness;
  direction = +world.z;
  result = solid;
}
```

Puis :

```
morphoia validate spacer.morph
morphoia compile spacer.morph -o build/spacer.mcir.json
```

Le fichier doit être accepté. Cela démontre que les symboles, unités simples, sketch et opération P1 sont représentables. Cela ne démontre pas que le disque a été extrudé par un noyau.

Ajouter une contrainte de domaine

Les attributs peuvent conserver des limites :

```
parameter diameter: length = 20 mm [
  min = 5 mm,
  max = 100 mm,
  status = observed,
  evidence = "drawing:D1"
];
```

Le compilateur conserve ces attributs. Le validateur actuel ne vérifie pas encore que la valeur se trouve entre min et max.

Tutoriel 3 - Comprendre une erreur d'unité

Remplacer temporairement :

```
parameter diameter: length = 20 mm;
```

par :

```
parameter diameter: length = 20 deg;
```

Valider :

```
morphoia validate spacer.morph
```

Le diagnostic attendu contient :

```
MORPH-E110: parameter 'diameter' expects length, got angle
```

Pour obtenir une structure JSON :

```
morphoia validate spacer.morph --json
```

Corriger l'unité en mm avant de poursuivre.

Tutoriel 4 - Déclarer une référence persistante

Après une feature body, ajouter :

```
reference top_face: face = select(
  body.faces,
  role = "cap.end",
  normal = world.z,
  cardinality = unique
);
```

Cette requête satisfait les quatre contrôles actuels : source, rôle, signature géométrique et cardinalité.

Observer l'échec volontaire

Supprimer `cardinality = unique` puis relancer la validation. Le résultat contient MORPH-E131.

Ajouter ensuite la cardinalité, mais supprimer `normal = world.z`. Le résultat contient MORPH-E134, car aucune preuve géométrique ou d'adjacence ne distingue la face.

Ces règles ne résolvent pas magiquement le Topological Naming Problem. Elles interdisent seulement les choix silencieux dans le contrat source.

Tutoriel 5 - Détecter un cycle de dépendances

Le fragment suivant est invalide :

```
morphoia 0.1;

model Cyclic [profile = "P1"] {
  parameter a: length = b;
  parameter b: length = a;
}

morphoia validate cyclic.morph
```

Le validateur produit MORPH-E103 et affiche le chemin du cycle. Les cycles de features sont interdits. À terme, les cycles algébriques d'un solveur d'esquisse seront traités dans un hypergraphe séparé ; aucun solveur n'est encore connecté.

Tutoriel 6 - Utiliser l'API Python

```
from pathlib import Path

from morphoia import canonical_json, compile_document, validate_source

source = Path("examples/mounting_plate.morph").read_text(encoding="utf-8")
validation = validate_source(source)

for diagnostic in validation.diagnostics:
    print(diagnostic.severity.value, diagnostic.code, diagnostic.message)

if not validation.ok or validation.document is None:
    raise SystemExit(1)

ir = compile_document(validation.document)
print("hash:", ir["semantic_sha256"])
Path("build/from_python.mcir.json").parent.mkdir(exist_ok=True)
Path("build/from_python.mcir.json").write_text(
    canonical_json(ir),
    encoding="utf-8",
)
```

`validate_source` capture les erreurs lexicales et syntaxiques dans un `ValidationResult`. `parse` est plus bas niveau et lève directement une exception.

Tutoriel 7 - Tester les trois états d'une référence

Le résolveur Python peut être utilisé sans noyau pour tester le contrat :

```
from morphoia.topology import (
    EntityCandidate,
    ReferenceQuery,
    ResolutionStatus,
    resolve_reference,
)

query = ReferenceQuery(
    kind="face",
    producer="blank",
    role="cap.end",
    signature=((("normal", "+Z"),)),
)

face_a = EntityCandidate(
    identity="face-a",
    kind="face",
    producer="blank",
    role="cap.end",
    signature=((("normal", "+Z"),)),
)

assert resolve_reference(query, []).status is ResolutionStatus.MISSING
assert resolve_reference(query, [face_a]).status is ResolutionStatus.UNIQUE

face_b = EntityCandidate(
    identity="face-b",
    kind="face",
    producer="blank",
    role="cap.end",
    signature=((("normal", "+Z"),)),
)

ambiguous = resolve_reference(query, [face_b, face_a])
assert ambiguous.status is ResolutionStatus.AMBIGUOUS
assert [item.identity for item in ambiguous.candidates] == ["face-a", "face-b"]
```

L'ordre déterministe des candidats facilite le diagnostic. Il ne choisit pas face-a à la place de l'utilisateur.

Tutoriel 8 - Vérifier le déterminisme du prototype

```
from pathlib import Path

from morphoia import compile_document, validate_source

source = Path("examples/mounting_plate.morph").read_text(encoding="utf-8")
first = validate_source(source)
second = validate_source(source)

assert first.ok and first.document is not None
assert second.ok and second.document is not None

first_ir = compile_document(first.document)
second_ir = compile_document(second.document)
assert first_ir["semantic_sha256"] == second_ir["semantic_sha256"]
```

Cette vérification porte sur deux compilations avec la même implémentation. Le gate de deux implémentations indépendantes n'est pas encore atteint.

Tutoriel 9 - Utiliser une transaction en mémoire

```
from morphoia import RuntimeStore

initial_graph = {"format": "example", "value": 1}
store = RuntimeStore(initial_graph)

transaction = store.begin()
transaction.graph["value"] = 2
revision = transaction.commit("change value")

assert revision.number == 1
assert store.current.graph["value"] == 2
assert store.undo().graph["value"] == 1
assert store.redo().graph["value"] == 2
```

Un validateur peut interrompre atomiquement le commit :

```
transaction = store.begin()

def reject(_graph):
    raise ValueError("invalid geometry")

try:
    transaction.commit("invalid change", validators=(reject,))
except ValueError:
    pass

assert store.current.number == 1
```

Ce magasin est volontairement en mémoire. Il démontre atomicité, undo/redo et contrôle optimiste ; il n'exécute ni géométrie ni scénario .morph.

Tutoriel 10 - Enregistrer preuve, décision et perte

```
from decimal import Decimal

from morphoia.losses import LossCategory, LossRecord, LossRegister, LossSeverity
from morphoia.provenance import Decision, DecisionStatus, Evidence, EvidenceKind

evidence = Evidence(
    identity="e1",
    kind=EvidenceKind.DRAWING,
    source_uri="urn:morphoia:drawing:1",
    source_sha256="a" * 64,
    locator="view:front/dimension:12",
)

decision = Decision(
    identity="d1",
    subject="width",
    status=DecisionStatus.ACCEPTED,
    evidence_ids=(evidence.identity,),
    confidence=Decimal("0.99"),
    decided_by="rule:explicit-dimension",
)

losses = LossRegister(
    operation="export",
    source_backend="canonical",
    target_backend="glTF",
    records=(
        LossRecord(
            code="MORPH-L001",
            category=LossCategory.PARAMETRIC_HISTORY,
            severity=LossSeverity.ERROR,
            subject="model",
            message="glTF does not preserve the construction history",
        ),
    ),
)

assert decision.status is DecisionStatus.ACCEPTED
assert losses.blocks_commit
```

Ces primitives ne sont pas encore reliées automatiquement au parser, à l'IR ou à la CLI. Elles rendent toutefois testables deux exigences de Phase 1 : aucune inférence acceptée sans preuve, aucune perte critique silencieuse.

FAQ

MORPHOIA est-il déjà un nouveau standard CAO ?

Non. MORPHOIA 0.1 est un candidat expérimental. La Phase 1 interdit de le présenter comme standard avant preuves de portabilité, conformité et valeur industrielle.

Pourquoi le fichier dit-il 0.1 alors que la CLI affiche 0.2.0-dev ?

0.1 est la version de la façade .morph et de l'IR. 0.2.0-dev est la version de développement du paquet Python. Le parser n'accepte actuellement que morphoia 0.1;.

La commande `compile` produit-elle un modèle CAO ?

Non. Elle produit un graphe JSON canonique. Aucun noyau B-rep n'est exécuté et aucun STEP n'est écrit.

Puis-je ouvrir le JSON dans FreeCAD, CATIA, NX ou SolidWorks ?

Non directement. Les adaptateurs et exports correspondants sont des cibles. Le JSON sert actuellement au débogage, aux tests et aux échanges internes du POC.

Le dépôt contient-il Open CASCADE ?

Non. Il contient un contrat abstrait de backend et un backend JSON. Un adaptateur OCCT devra être ajouté sans exposer de types OCCT dans le graphe canonique.

Pourquoi ne pas écrire directement du CadQuery ?

CadQuery reste une brique de référence et un backend/prototype potentiel. La façade expérimentale teste des propriétés supplémentaires : absence d'I/O arbitraire, diagnostics structurés, provenance, profils, tolérances séparées et ambiguïté topologique explicite. Son maintien dépendra de benchmarks contre CadQuery ; elle sera abandonnée si aucun avantage mesurable n'est démontré.

Pourquoi STEP/AP242 n'est-il pas simplement remplacé ?

Il ne doit pas l'être. Le graphe est ancré dans ISO 10303-42/-55/-108/-109/-111/ -112/-113 et AP242. La syntaxe `.morph` n'est qu'une façade candidate plus simple à écrire et à générer. Les mappings exacts et le round-trip restent à implémenter.

Les opérations `extrude` et `fillet` fonctionnent-elles ?

Le parser les reconnaît et le validateur contrôle leur appartenance au profil. Il ne vérifie pas encore toutes leurs signatures et ne les exécute pas. « Reconnue » ne signifie donc pas « géométriquement implémentée ».

Pourquoi mon feature incomplet peut-il être déclaré valide ?

Les contrats détaillés d'opération ne sont pas encore branchés au validateur. La validation actuelle couvre uniquement un sous-ensemble statique. Cette lacune fait partie du travail G0.

Quels profils existent ?

- P1 : opérations prismatiques et tournées de base ;
- P2 : P1 plus thread, matériaux et PMI cibles ;
- P3 : assemblages et configurations cibles ;
- P4 : géométrie avancée.

Dans le prototype, les profils contrôlent surtout la liste des noms d'opérations. Aucun profil n'est encore exécuté par un noyau.

Pourquoi `cardinality = unique` est-il obligatoire ?

Parce que sélectionner silencieusement la première face parmi plusieurs candidats peut modifier une pièce. Une référence doit être unique, manquante ou ambiguë. Le dernier état nécessite une décision ou une meilleure requête.

Puis-je utiliser `cardinality = many` ?

Le modèle cible prévoit des ensembles typés, mais le validateur de références persistantes actuel exige unique ou one. Les références multi-entités devront recevoir un contrat séparé avant d'être acceptées.

Les UUID sont-ils obligatoires ?

Pas dans le prototype. En leur absence, le compilateur dérive un UUIDv5 du namespace et du chemin. Comme un renommage modifie ce chemin, un UUID explicite est fortement recommandé pour tout actif appelé à survivre aux éditions et round-trips.

Le hash sémantique est-il définitif ?

Non. Il est déterministe dans l'implémentation actuelle, mais le contrat de canonicalisation peut évoluer en 0.x. Le gate exige encore une seconde implémentation indépendante et une bijectivité complète.

Les imports sont-ils téléchargés ?

Non. Le parser conserve une URI ou un chemin et un alias. Aucun resolver, allowlist, digest ou téléchargement n'est actif.

Les scénarios `validate` sont-ils exécutés ?

Non. Ils sont parsés et abaissés dans l'IR. Leur exécution transactionnelle après modification de paramètres est une cible essentielle de conformité comportementale.

MORPHOIA possède-t-il déjà undo/redo et des transactions ?

Oui, sous forme d'un `RuntimeStore` en mémoire testé unitairement. Il gère commit atomique, rollback, undo/redo et conflits optimistes sur le graphe JSON. Il n'est pas encore persistant et n'est pas raccordé à un noyau ou aux scénarios `.morph`.

Le registre des pertes est-il déjà utilisé par les exports ?

Le type `LossRegister` et sa règle de blocage existent, mais aucun export CAO n'est encore implémenté. Son intégration aux adaptateurs reste une cible.

MORPHOIA utilise-t-il déjà l'IA ou le GPU ?

Non. Les documents définissent les pipelines visés, mais le code présent est un lexer, parser, validateur partiel, compilateur JSON et contrat de référence.

Peut-on générer des fichiers `.morph` avec un LLM ?

Oui à titre expérimental, à condition de toujours les passer au validateur et de ne pas confondre validité syntaxique avec exactitude mécanique. La cible de Phase 2 est au moins 98 % de sorties syntaxiquement valides avec décodage contraint, mais elle n'est pas encore démontrée.

Quelle est la licence ?

Le dépôt est distribué sous licence MIT, conformément au fichier `LICENSE` choisi lors de la création du dépôt GitHub. Les standards tiers, SDK commerciaux, corpus et marques restent soumis à leurs propres droits. Une éventuelle politique distincte pour la spécification, les données ou la marque exigera une décision et une revue juridique séparées.

Comment proposer un composant nouveau ?

Il faut d'abord remplir le [Component Justification Record](decisions/CJR-TEMPLATE.md), comparer les solutions existantes, préenregistrer la métrique et démontrer le manque ou l'avantage. Une amélioration esthétique d'API ne suffit pas.

Quels sont les gates avant une version 1.0 ?

Gate	Critère principal	Statut
G0	Draft, implémentation de référence, 100 cas golden	En cours
G1	Deux backends P1/P2, B-rep $\geq 99\%$, scénarios $\geq 95\%$	Non atteint
G2	Conformité publique et implémentation indépendante	Non atteint
G3	Gain humain médian $\geq 40\%$ sur deux pilotes	Non atteint
G4	Gouvernance externe et politique brevets après G1-G3	Interdit avant preuves

Où continuer ?

- [Guide utilisateur](user-guide.md)
- [Guide développeur](developer-guide.md)
- [Spécification candidate](language-specification.md)
- [Grammaire EBNF](grammar.ebnf)
- [Exigences et gates](requirements.md)

Feuille de route, gouvernance et adoption

Statut de ce document

Ce chapitre de la Phase 2 décrit une trajectoire conditionnelle. Il ne transforme pas la façade textuelle MORPHOIA 0.1 en standard. La Phase 1 impose trois conditions cumulatives avant toute stabilisation normative : exécution qualifiée du profil P1/P2 sur au moins deux backends indépendants, publication d'une suite de conformité et mesure d'un bénéfice industriel. Jusqu'à leur satisfaction, la grammaire, l'IR JSON et les API restent expérimentales.

Principes de conduite

1. Réutiliser avant de développer : STEP/AP242 porte l'autorité d'échange, OCCT le backend ouvert de référence et les solveurs existants sont évalués avant tout remplacement.
2. Limiter le périmètre par profils : aucune opération P4 ne bloque la qualification P1.
3. Mesurer avant d'optimiser : chaque composant nouveau est précédé d'un Component Justification Record (CJR) et d'un benchmark reproductible.
4. Ne jamais masquer une perte, une réparation géométrique, une ambiguïté ou une hypothèse issue de l'IA.
5. Garder la géométrie exacte et les décisions normatives sur CPU tant qu'une voie accélérée n'a pas démontré une équivalence bornée.

Profils de conformité proposés

Profil	Périmètre	Gate principal
P0 - syntaxe	parsing, types, unités, DAG, diagnostics, IR canonique	corpus syntaxique public et hash stable
P1 - pièce mécanique	croquis 2D, extrude, cut, revolve, hole, fillet, chamfer, pattern, mirror, propriétés	OCCT et un second backend, scénarios d'édition $\geq 95\%$
P2 - MBD	paramètres, datums, PMI sémantique, tolérances, matériaux, AP242	association PMI et round-trip $\geq 99\%$
P3 - assemblage	occurrences, placements rigides, configurations, mates simples	deux CAO et aucun degré de liberté caché
P4 - géométrie avancée	sweep, loft, blend, shell, draft, lois variables	qualification opération par opération

Un produit peut annoncer plusieurs profils, mais doit publier le manifeste exact de ses capacités. Un export partiel produit obligatoirement un registre de pertes.

Étapes et gates

Fondation - mois 0 à 3

Livrables :

- gel des exigences issues de la Phase 1 et matrice de traçabilité ;
- étude de liberté d'exploitation, politique brevets et contrats de données ;
- définition précise de P0, P1 et P2 ;
- API backend et manifeste de capacités ;
- 100 cas golden, microcas dégénérés et propriétés attendues ;
- procédure ADR/RFC/CJR et politique de sécurité.

Gate : aucune sémantique nouvelle n'est acceptée sans mapping STEP ou CJR approuvé.

Budget indicatif : 0,4 à 0,8 M EUR.

0.1 POC - mois 3 à 12

Livrables :

- parseur, validateur, IR canonique et SDK Python ;
- runtime C++ minimal avec ABI C versionnée ;
- backend OCCT/XDE, import/export STEP et propriétés de validation ;
- 8 à 12 opérations P1 ;
- résolveur de références à trois états ;
- 500 à 2 000 pièces et au moins cinq scénarios d'édition par pièce ;
- premier rapport public de divergences et de pertes.

Gate : validité B-rep $\geq 98\%$, zéro ambiguïté choisie silencieusement, reproduction du hash sémantique sur 100 exécutions et démonstration de la réversibilité des transactions.

Budget cumulé : 1,8 à 4,8 M EUR.

0.3 Alpha - mois 12 à 18

Livrables :

- backend FreeCAD headless ou autre deuxième implémentation effectivement indépendante ;
- comparaison différentielle OCCT/second backend ;
- pipeline IA multimodal initial avec décodage contraint et abstention ;
- accélération CUDA des tâches denses, export Blender/glTF ;
- deux pilotes industriels ;
- documentation SDK, notebooks et laboratoire de compatibilité.

Gate : deux backends exécutent le même sous-profil, les écarts sont expliqués et les pilotes montrent que les données sont légalement utilisables.

Budget cumulé : 4 à 8 M EUR.

0.5 MVP vertical - mois 18 à 30

Le MVP cible volontairement les pièces usinées prismatiques et tournées reconstruites depuis des plans, STEP sans historique ou scans simples. Il couvre P1 et un sous-ensemble borné P2.

Critères de passage :

- validité B-rep $\geq 99\%$;
- dimensions explicitement cotées : 100 % correctes ou abstention ;
- scénarios d'édition réussis $\geq 95\%$;
- round-trip AP242 avec propriétés de validation $\geq 99\%$;
- détection hors profil $\geq 95\%$;
- gain médian de temps humain $\geq 40\%$;
- coût total par pièce inférieur à 50 % du coût manuel de référence ;
- au moins 10 000 pièces de test, séparées par famille, fournisseur et date.

Budget cumulé : 7 à 19 M EUR, réserve de risque comprise.

0.8 Bêta - mois 30 à 42

Livrables : P3 simple, matrice de versions CAO, suite de conformité publique, télémétrie consentie, processus de vulnérabilités, cinq à dix clients et premier événement d'interopérabilité indépendant.

Gate : deux implémentations indépendantes et plusieurs organisations reproduisent les résultats du profil candidat sans accès à un oracle privé.

Budget cumulé : 15 à 35 M EUR.

1.0 - mois 42 à 60

La version 1.0 n'est autorisée que si les trois gates de la Phase 1 sont franchis. Elle doit comprendre :

- P1/P2 qualifiés sur deux backends ;
- quatre intégrations CAO maintenues avec versions explicites ;
- migration de schéma documentée et LTS d'au moins 24 mois ;
- conformance suite et corpus de référence publiés ;
- gouvernance externe effective ;
- preuve économique indépendante et reproductible.

Budget cumulé : 30 à 90 M EUR. Une fondation industrielle à grande échelle peut ajouter 8 à 30 M EUR.

Organisation cible

Organes

- Comité technique : architecture, runtime, SDK et qualité.
- Comité normes et conformité : mappings ISO, profils et certification.
- Comité données, propriété intellectuelle et sécurité : corpus, licences, confidentialité,

brevets et incidents.

- Mainteneurs de backends : qualification de chaque noyau et version CAO.
- Conseil utilisateurs industriels : priorités, critères d'acceptation et retour d'usage.

Aucun fournisseur de noyau ne doit disposer seul d'un veto sur le modèle neutre. Une décision normative exige au minimum deux implémentations ou une preuve formelle qu'elle ne dépend pas d'un backend particulier.

Processus de décision

- Une ADR enregistre les décisions irréversibles ou structurantes.
- Une RFC publique précède les évolutions du modèle canonique.
- Un CJR est obligatoire pour tout composant inventé ou toute dépendance stratégique.
- Les modifications incompatibles suivent SemVer et fournissent une migration exécutable.
- Les minutes, votes, conflits d'intérêts et résultats de conformité sont publics dès

l'ouverture du projet.

- Les cas de test deviennent normatifs seulement après revue croisée et reproductibilité.

Cycle de publication

Les séries 0.x peuvent évoluer rapidement, mais chaque artefact doit déclarer sa version. Les versions 1.x suivent un cycle prévisible : proposition, période de commentaires, release candidate, campagne de conformité, publication, puis support LTS. Une extension ne peut redéfinir silencieusement une opération du coeur.

Licences et politique de propriété intellectuelle

Le dépôt public a été créé sous licence MIT par Olivier Ami. Le code, les spécifications et les rapports contenus dans cette version suivent donc le fichier LICENSE. Cette licence ne s'applique pas automatiquement aux standards tiers, SDK commerciaux, jeux de données, modèles ou marques cités.

Avant une version normative et avant l'arrivée de contributions externes substantielles, une revue juridique doit comparer le maintien de MIT à une politique différenciée, par exemple :

- Apache License 2.0 pour les futures versions du runtime, du SDK, des outils et du validateur,

notamment pour sa clause de licence de brevets ;

- CC BY 4.0 pour de futures éditions de la spécification et des guides ;
- licence de données séparée et vérifiée pour chaque corpus ;

- licences commerciales pour les adaptateurs propriétaires, jeux de données sensibles ou modèles qui ne peuvent pas être ouverts ;
- politique de marque distincte pour le nom et le label de conformité MORPHOIA.

Une modification future ne retire pas rétroactivement les droits déjà accordés sous MIT. Le choix doit être précédé d'une revue juridique d'OCCT, FreeCAD, Blender, CUDA/OptiX, des SDK commerciaux, des brevets et des droits sur les données. Les contributions peuvent suivre un DCO ou un CLA ; le mécanisme retenu doit préserver l'attribution tout en protégeant la capacité de standardisation et de gestion des brevets.

Stratégie d'adoption

1. Résoudre un périmètre étroit avant de revendiquer l'universalité.
2. S'intégrer dans les CAO existantes ; ne pas demander aux ingénieurs de les remplacer.
3. Afficher pour chaque résultat la source, l'hypothèse, la perte, le diff et les scénarios d'édition.
4. Publier des connecteurs de lecture et des validateurs avant les outils de création.
5. Organiser des plugfests avec CAx-IF, intégrateurs, éditeurs, fabricants et métrologues.
6. Proposer les profils matures à une organisation de standardisation seulement après deux implémentations indépendantes.

Le bénéfice industriel attendu est la réduction des reconstructions manuelles, une meilleure traçabilité des décisions IA, un archivage moins dépendant d'un éditeur, des migrations CAO mesurables et une réutilisation plus sûre des actifs mécaniques. Ces bénéfices restent des hypothèses tant qu'ils ne sont pas mesurés sur les pilotes aveugles définis ci-dessus.

Coûts, risques et analyse de faisabilité industrielle

Portée et méthode

Les valeurs suivantes sont des ordres de grandeur 2026 exprimés hors taxes. Elles couvrent les équipes, l'infrastructure, les licences de développement, la qualité, le juridique et une réserve de risque. Elles ne constituent ni devis ni promesse de calendrier. L'intervalle est large parce que l'accès aux SDK propriétaires, aux corpus industriels et aux experts STEP domine l'incertitude.

Estimation par phase

Phase	Durée	Charge indicative	Coût cumulé
Fondation	0-3 mois	4-8 ETP	0,4-0,8 M EUR
POC 0.1	3-12 mois	8-15 ETP.an	1,8-4,8 M EUR
Alpha 0.3	12-18 mois	18-30 personnes	4-8 M EUR
MVP 0.5	18-30 mois	35-60 ETP.an cumulés	7-19 M EUR
Bêta 0.8	30-42 mois	45-80 personnes	15-35 M EUR
Version 1.0	42-60 mois	150-300 ETP.an cumulés	30-90 M EUR

Après le MVP, le maintien des versions CAO, SDK, modèles, corpus et règles de conformité représente environ 18 à 30 % du coût initial par an, soit typiquement 15 à 35 personnes. Une réserve de 20 à 35 % est recommandée jusqu'au MVP.

Équipe MVP indicative

- 6 à 10 spécialistes géométrie exacte, STEP, PMI et solveurs ;
- 5 à 8 ingénieurs SDK et adaptateurs CAO ;
- 6 à 10 ingénieurs IA, vision et données ;
- 3 à 5 ingénieurs GPU et plateforme ;
- 5 à 8 spécialistes QA, conformité, métrologie et données de référence ;
- 3 à 5 personnes produit, sécurité, propriété intellectuelle et opérations.

La concentration de compétences rares est un risque en soi : STEP procédural, PMI sémantique, robustesse B-rep et traduction native disposent de viviers limités. Les revues externes et partenariats universitaires doivent donc être budgétés dès la fondation.

Coûts directs à ne pas sous-estimer

1. Licences Parasolid, ACIS, CGM et SDK de traduction, avec environnements de test séparés.
2. Licences et automatisation des versions CATIA, NX, Creo, SolidWorks, Inventor, Fusion 360 et Rhino.
3. Construction et annotation d'un corpus légal contenant historique, PMI, assemblages et scénarios d'édition.
4. Laboratoire matériel multi-OS, GPU NVIDIA/AMD/Intel et CI de longue durée.
5. Audits sécurité, propriété intellectuelle, export control et conformité des données.
6. Participation aux groupes ISO, LOTAR, PDES/CAX-IF et événements d'interopérabilité.
7. Support LTS, migration des schémas et maintenance des backends lors des mises à jour annuelles des éditeurs.

Registre des risques

Risque	Probabilité	Impact	Signal précoce	Mesure de réduction
Périmètre universel prématuré	élevée	critique	ajout de P4 avant stabilité P1	profils versionnés et gates contractuels
Référence topologique cassée	élevée	critique	petite édition invalidant de nombreuses cibles	lignée, rôle, adjacence, signature et abstention
Divergence des noyaux	élevée	critique	volumes ou branches différents	tests différentiels, propriétés et pertes explicites
Round-trip natif irréalisable	élevée	élevée	historique propriétaire non récupéré	contrats bornés par format/version, pas de promesse universelle
Dataset illégal ou biaisé	moyenne	critique	sources sans droits ou benchmark trop facile	audit juridique, splits par famille, test aveugle industriel
Hallucination IA	élevée	critique	cotes plausibles sans preuve	preuves obligatoires, décodage contraint, abstention
Sur-ajustement au benchmark	moyenne	élevée	forte chute chez les pilotes	corpus secret, adversarial et multi-fournisseur
Dépendance SDK fournisseur	élevée	élevée	prix/ABI dicte la roadmap	isolation hors processus et deux backends minimum
GPU surestimé	élevée	moyenne	temps dominé par les booléens CPU	profilage de bout en bout avant chaque portage
Non-déterminisme numérique	moyenne	élevée	hash ou résultat instable	contrat numérique, versions verrouillées, répétitions
Explosion de matrice CAO	élevée	élevée	régressions à chaque version	versions supportées explicites et laboratoire CI
Sécurité des fichiers/plugins	moyenne	critique	accès réseau ou système imprévu	sandbox, quotas, processus isolés, signatures
Rejet par les ingénieurs	moyenne	critique	corrections répétées ou décisions opaques	UX de preuve, diff, revue et mesure du temps net
Litige brevet/licence	faible à moyenne	critique	revendication sur feature ou dataset	FTO, registre SBOM et revue juridique continue
Gouvernance capturée	moyenne	élevée	spécification dépendant d'un seul acteur	représentation équilibrée et implémentations indépendantes

Verrous scientifiques

Intention sous-déterminée

Une même B-rep ou photographie correspond à plusieurs historiques paramétriques valides. Il n'existe généralement pas de reconstruction unique de l'intention. MORPHOIA doit conserver les alternatives, la provenance et demander une décision lorsque les sources ne départagent pas les candidats.

Topological Naming Problem

La continuité d'identité après fusion, scission ou disparition ne peut pas être garantie dans tous les cas. Le résultat scientifique réaliste est une résolution explicite unique, ambiguous ou missing, avec scénarios de validation. Toute revendication de solution universelle serait trompeuse.

Équivalence procédurale

Deux noyaux peuvent construire des B-rep différentes mais fonctionnellement équivalentes. L'équivalence doit combiner propriétés, tolérances, PMI et comportement après modification ; la comparaison octet à octet ne suffit pas. La définition publique de cette équivalence est un verrou central.

Robustesse géométrique

Les booléens, offsets, blends, singularités et petits détails restent sensibles aux tolérances et algorithmes propriétaires. Une couche neutre ne supprime pas ces limites : elle les rend observables, reproductibles et comparables.

Validation IA

Les sorties hors distribution et les erreurs de calibration rendent impossible une autonomie générale sûre à court terme. La voie faisable est une IA de proposition avec exécution déterministe, preuves, seuils d'abstention et revue humaine.

B-rep exacte sur GPU

Aucun noyau B-rep général, exact, robuste et déterministe entièrement GPU ne peut aujourd'hui être considéré comme une brique prête à réutiliser. Le projet doit accélérer les tâches denses et conserver un chemin CPU de référence.

Verrous industriels

- accès contractuel aux formats natifs et droits de redistribution ;
- absence de corpus public représentatif avec PMI et historique ;
- qualification multi-version coûteuse des CAO ;
- confidentialité des pièces réelles et contraintes d'export ;
- adoption d'un profil ouvert par des acteurs aux intérêts divergents ;
- responsabilité lorsqu'une reconstruction alimente fabrication ou contrôle qualité ;
- preuve qu'un historique neutre reste modifiable avec le même sens sur plusieurs outils.

Critères d'arrêt

Le projet doit pouvoir s'arrêter ou réduire son périmètre si l'un des faits suivants est observé après un pilote correctement instrumenté :

- aucun second backend ne peut exécuter P1 sans dépendre de choix cachés du premier ;
- le gain médian de temps humain reste inférieur à 20 % malgré une précision qualifiée ;
- le taux d'édition réussie reste inférieur à 80 % sur le profil ciblé ;
- les droits nécessaires aux corpus ou SDK rendent une conformance publique impossible ;
- une dépendance propriétaire devient indispensable à la lecture du modèle neutre ;
- le coût par pièce reste supérieur au coût manuel sur les volumes visés.

Ces seuils sont des règles de gouvernance proposées, pas des conclusions déjà démontrées.

Bénéfices industriels à mesurer

- réduction du temps de reconstruction et de migration ;
- baisse du verrouillage fournisseur pour l'archivage et l'automatisation ;
- traçabilité des cotes, hypothèses et corrections IA ;
- détection plus tôt des pertes PMI et incohérences géométriques ;
- réutilisation des modèles pour simulation, fabrication, métrologie et visualisation ;
- comparaison objective des noyaux et traducteurs ;
- création d'un écosystème de backends et validateurs certifiables.

Le business case doit rapporter ces bénéfices au coût complet : licences, temps de revue, corrections, calcul, stockage, formation, incidents et maintenance. La métrique centrale est le temps humain net économisé par pièce acceptée, accompagné du taux d'abstention et du coût des erreurs évitées.

Références normatives et techniques de la Phase 2

Politique de citation

La bibliographie scientifique, industrielle, brevets, thèses, projets et logiciels étudiés est conservée dans le rapport de Phase 1. Le présent chapitre liste les sources directement opposables aux choix de Phase 2 et évite de dupliquer cette revue. Une date de consultation du 3 août 2026 s'applique aux pages web, sauf indication contraire.

Les textes ISO complets sont soumis aux conditions de leur éditeur. MORPHOIA cite les fiches officielles et ne reproduit pas leur contenu normatif.

ISO 10303 et STEP

1. ISO 10303-11:2004, *Industrial automation systems and integration - Product data representation and exchange - Part 11: Description methods: The EXPRESS language reference manual*. <https://www.iso.org/standard/38047.html>
2. ISO 10303-21, Implementation methods: Clear text encoding of the exchange structure. Catalogue ISO 10303.
3. ISO 10303-28, Implementation methods: XML representations of EXPRESS schemas and data. Catalogue ISO 10303.
4. ISO 10303-42, Integrated generic resource: Geometric and topological representation. Catalogue ISO 10303.
5. ISO 10303-55:2005, Integrated generic resource: Procedural and hybrid representation. <https://www.iso.org/fr/standard/38226.html>
6. ISO 10303-108:2005, *Integrated application resource: Parameterization and constraints for explicit geometric product models*. <https://www.iso.org/standard/34697.html>
7. ISO 10303-109, *Integrated application resource: Kinematic and geometric constraints for assembly models*. Catalogue ISO 10303.
8. ISO 10303-111:2007, *Integrated application resource: Elements for the procedural modelling of solid shapes*. <https://www.iso.org/standard/39561.html>
9. ISO 10303-112:2006, *Integrated application resource: Modelling commands for the exchange of procedurally represented 2D CAD models*. <https://www.iso.org/standard/39581.html>
10. ISO 10303-113:2025, Integrated application resource: Mechanical product features. <https://www.iso.org/standard/91389.html>
11. ISO 10303-203, *Application protocol: Configuration controlled 3D design of mechanical parts and assemblies*.
12. ISO 10303-214, *Application protocol: Core data for automotive mechanical design processes*.
13. ISO 10303-242:2025, édition 4, Application protocol: Managed model-based 3D engineering. <https://www.iso.org/fr/standard/84300.html>
14. ISO 10303-238, *Application protocol: Application interpreted model for computerized numerical controllers*.

Autres standards et formats

15. ISO 14739, *Document management - 3D use of Product Representation Compact (PRC) format*.

16. ISO 14306, *Industrial automation systems and integration - JT file format specification for 3D visualization*.

17. ISO 16739-1, *Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries*.

18. ASME Y14.5 et ISO GPS, spécifications dimensionnelles et tolérancement géométrique. Les éditions et profils applicables doivent être déclarés dans chaque implémentation.

19. Khronos Group, glTF 2.0 Specification.

<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

20. Alliance for OpenUSD, OpenUSD documentation and specifications.

<https://openusd.org/release/index.html>

21. buildingSMART International, IFC specifications database.

<https://technical.buildingsmart.org/standards/ifc/ifc-schema-specifications/>

Noyaux, CAO et SDK

22. Open CASCADE, OCCT documentation.

<https://dev.opencascade.org/doc/overview/html/>

23. FreeCAD, Developer documentation.

<https://freecad.github.io/SourceDoc/>

24. CadQuery, Introduction et Selectors reference.

<https://cadquery.readthedocs.io/en/latest/intro.html> ; <https://cadquery.readthedocs.io/en/stable/selectors.html>

25. Onshape, FeatureScript documentation.

<https://cad.onshape.com/FsDoc/index.html>

26. OpenSCAD, User Manual.

<https://opencad.org/documentation.html>

27. Siemens Digital Industries Software, documentation publique Parasolid et JT.

28. Dassault Systèmes, documentation publique ACIS, CATIA et 3D InterOp.

29. Siemens, PTC, Dassault Systèmes, Autodesk et McNeel, documentations publiques de NX,

Creo, SolidWorks, Fusion 360, Inventor et Rhino. Chaque adaptateur MORPHOIA doit citer la version exacte de l'API réellement qualifiée.

30. Tech Soft 3D HOOPS Exchange, CAD Exchanger et Datakit, documentations SDK publiques.

Les descriptions de Parasolid XT, ACIS SAT et formats CAO natifs ne constituent pas une autorisation de redistribution. Les licences contractuelles et versions d'API priment sur les pages marketing.

Compilateurs et exécution

31. LLVM Project, LLVM Language Reference Manual.

<https://llvm.org/docs/LangRef.html>

32. LLVM Project, MLIR Language Reference.

<https://mlir.llvm.org/docs/LangRef/>

33. LLVM Project, Defining Dialects et Defining Dialect Operations.

<https://mlir.llvm.org/docs/DefiningDialects/> ; <https://mlir.llvm.org/docs/DefiningDialects/Operations/>

34. OpenXLA Project, documentation StableHLO et XLA.

<https://openxla.org/>

35. ONNX, Open Neural Network Exchange documentation.

<https://onnx.ai/onnx/>

36. ONNX Runtime, documentation d'exécution et fournisseurs matériels.

<https://onnxruntime.ai/docs/>

IA, vision et GPU

37. Vaswani et al., Attention Is All You Need, NeurIPS 2017.

38. Dosovitskiy et al., An Image is Worth 16x16 Words, ICLR 2021.

39. Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020.

40. Qi et al., PointNet et PointNet++, 2017.

41. NVIDIA, CUDA C++ Programming Guide.

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

42. NVIDIA, OptiX Programming Guide.

<https://raytracing-docs.nvidia.com/optix8/guide/index.html>

43. Khronos Group, Vulkan Specification.

<https://registry.khronos.org/vulkan/>

44. AMD, ROCm documentation. <https://rocm.docs.amd.com/>

45. Khronos Group, OpenCL specifications. <https://www.khronos.org/opencl/>

46. Khronos Group, SYCL specifications. <https://www.khronos.org/sycl/>

47. Microsoft, DirectML documentation.

<https://learn.microsoft.com/windows/ai/directml/dml>

48. NVIDIA, documentation Warp, Kaolin et PyTorch3D ; OpenVDB et NanoVDB pour les volumes.

Les études CAD génératives et de reconstruction - notamment SketchGraphs, DeepCAD, Fusion 360 Gallery et les travaux B-rep/points récents - sont discutées et sourcées dans la Phase 1. Leur utilisation future exige une vérification des licences, des splits de données, des métriques et de la reproductibilité des modèles.

Interopérabilité et archivage

49. CAX Implementor Forum, recommandations et campagnes d'interopérabilité STEP.

<https://www.cax-if.org/>

50. LOTAR International, pratiques d'archivage long terme des données CAO.

<https://lotar-international.org/>

51. NIST, travaux sur les propriétés de validation et l'interopérabilité MBD.

52. DMSC, Quality Information Framework (QIF).

<https://qifstandards.org/>

Références propres à MORPHOIA

53. Olivier Ami, MORPHOIA - Phase 1 : État de l'art et étude de faisabilité, 2026.

54. MORPHOIA, docs/phase2/requirements.md, exigences héritées et critères de preuve.

55. MORPHOIA, docs/phase2/decisions/, ADR et modèle CJR.

56. MORPHOIA, schemas/, encodages expérimentaux de l'IR, des backends et des pertes.

Toute source ajoutée ultérieurement doit indiquer son éditeur, sa version ou date, son URL ou identifiant pérenne, sa licence lorsque pertinente et l'exigence MORPHOIA qu'elle soutient.

ADR-0001 - Coeur STEP-centrique

- Statut : accepté pour expérimentation
- Date : 2026-08-03
- Auteur du projet : Olivier Ami

Contexte

AP242 et ISO 10303-42/-55/-108/-109/-111/-112/-113 couvrent déjà produit, géométrie, procédures, paramètres, contraintes, esquisses et features. Créer une ontologie concurrente violerait GOV-001 et SEM-002.

Décision

Le modèle abstrait MORPHOIA réutilise ces concepts. Les nouveaux concepts doivent être namespacés, accompagnés d'un mapping ou d'un écart formel, et soumis à une Component Justification Record.

Conséquences

- AP242 devient l'autorité d'échange mécanique.
- Le JSON 0.1 n'est pas une nouvelle autorité sémantique.
- Les schémas EXPRESS officiels devront être analysés avant de figer des noms d'entités.
- Une dépendance à un SDK STEP unique est interdite.

ADR-0002 - Façade textuelle expérimentale

- Statut : accepté sous condition
- Date : 2026-08-03
- Auteur du projet : Olivier Ami

Contexte

EXPRESS spécifie des données mais n'est pas un langage d'exécution. FeatureScript est lié à Onshape. CadQuery réutilise Python généraliste. MLIR n'a pas de sémantique mécanique. Aucune option ne réunit syntaxe bornée, génération IA, identité explicite, provenance et exécution multi-backend.

Décision

Expérimenter une façade `.morph` déclarative, sans effets de bord et compilée vers le graphe canonique. Elle reste non normative en 0.x.

Critères de maintien

- 100 % de round-trip sémantique du profil ;
- deux parseurs indépendants produisant les mêmes hashes ;
- gain significatif de validité IA face à CadQuery et MLIR générique ;
- réduction préenregistrée du temps de correction humaine ;
- absence d'I/O et terminaison démontrée.

Condition d'abandon

Si ces avantages ne sont pas démontrés, conserver uniquement le graphe STEP-centrique et l'API Python typée.

ADR-0003 - OCCT comme backend ouvert de référence

- Statut : accepté
- Date : 2026-08-03
- Auteur du projet : Olivier Ami

Contexte

OCCT fournit B-rep exacte, opérations géométriques, OCAF, XDE et STEP. Réécrire un noyau demanderait plusieurs centaines d'ETP-an et n'apporterait pas de bénéfice démontré au POC.

Décision

OCCT est le premier backend exact. FreeCAD apporte un oracle ouvert. Parasolid, ACIS, CGM ou C3D sont optionnels sous licence.

Conséquences

- aucune classe OCCT dans le schéma canonique ;
- adaptateur versionné ;
- tests adversariaux et différentiels ;
- healing toujours déclaré ;
- aucun projet de noyau B-rep GPU généraliste dans la roadmap initiale.

CJR-XXXX - Nom du composant

Métadonnées

- Statut : proposé / testé / accepté / rejeté
- Propriétaire :
- Date :
- Exigences concernées :

Besoin non couvert

Décrire le besoin, le profil, le corpus et la métrique.

Solutions existantes évaluées

Solution et version	Couverture	Performance	Licence	Pérennité	Résultat
---------------------	------------	-------------	---------	-----------	----------

Protocole reproductible

Décrire données, matériel, versions, warm-up, répétitions, statistique et seuil préenregistré.

Résultat

Montrer l'impossibilité ou l'avantage mesuré, avec intervalle d'incertitude.

Interopérabilité

Mapping STEP, pertes, fallback et impact sur les backends.

Coût et risques

Développement, maintenance, sécurité, IP et dépendances.

Stratégie de remplacement

Expliquer comment retirer le composant sans rendre les archives illisibles.

Décision et condition d'abandon

Documenter la décision, les approbateurs, la date et le seuil de réévaluation.